
Learning to Search, Searching to Learn: A Closed-Loop Framework for Large-Scale Vehicle Problems

Anonymous Author(s)

Affiliation

Address

email

Abstract

大规模车辆路径规划问题（VRP）需要在可扩展性、全局结构建模与解质量之间取得平衡。现有神经求解器在扩展到超大规模实例时，往往依赖分解、局部候选或阶段化决策来控制计算成本，从而削弱对跨区域、跨路线全局结构的刻画；与此同时，经典启发式搜索虽能显著提升最终解，但在多数方法中仍主要作为神经预测之后的一次性后处理模块，搜索过程中暴露出的结构信息既不会在同一次推理中反馈给模型，也很少被系统性地转化为后续训练监督。为此，本文提出 **LSL** (**L**earning to **S**earch, **S**earching to **L**earn)，一个面向大规模 VRP 的神经-启发式闭环框架。LSL 不直接预测完整可行解，而是在稀疏候选图上学习对启发式搜索友好的全局结构先验；在推理阶段，模型根据当前解的结构状态持续更新该先验，并与启发式搜索后端交替执行多轮精炼；在训练阶段，LSL 将启发式搜索得到的多条高质量解投影为逐行软目标，使搜索结果不仅改进当前实例的求解，还能持续提升后续学习。基于这一设计，LSL 在固定候选宽度下实现了近似线性的计算与显存扩展，并能在单张消费级 GPU 上直接处理最高 100K 节点的实例。在大规模 VRP 基准上的实验表明，LSL 在解质量、推理效率与可扩展性之间取得了优异平衡，并在多个设置下达到最先进的结果。

1 Introduction

车辆路径规划问题（Vehicle Routing Problems, VRP）是组合优化中最经典也最重要的问题族之一，在物流配送、交通运输、仓储调度与供应链管理等场景中具有广泛应用 [???]。然而，VRP 及其典型变体在计算上通常是 NP 难的，随着实例规模增大，精确求解很快变得不可承受 [?]。因此，在工业界与运筹学界，研究者长期依赖 Lin-Kernighan-Helsgaun (LKH) 类局部搜索、混合遗传算法等经典启发式求解器来获得高质量近似解 [?]。这类方法在实践中极为强大，但其性能往往高度依赖人工设计的候选结构、局部算子与问题经验。近年来，神经组合优化（Neural Combinatorial Optimization, NCO）提供了另一条思路，即让模型直接从数据中学习求解策略或结构先验，而不是手工指定所有决策规则 [?????]。尽管这一方向在小到中等规模实例上取得了显著进展，如何在超大规模 VRP 上同时保持可扩展性、全局结构感知与高解质量，仍然远未解决。

第一个核心困难在于，当规模增大到数千、数万乃至十万节点时，学习型求解器如何在可控计算预算下仍然保留对全局结构的建模能力。近年的一些工作尝试通过更强的构造式求解器、更好的归纳偏置或测试时适配来提升跨尺度泛化能力 [??????]；另一些工作则依赖分解、层次化求解、局部候选限制或阶段化修正来扩大可处理规模 [??????]。这些方向都显著推动了大规模 VRP 研究，但也反映出一个共同张力，即为了控制复杂度，求解器越来越倾向于将决策限制在局部邻域、局部子问题或分阶段流程中，而真正决定整体质量的跨区域、跨路线长程耦合关系却更容易在这一过程中被削弱。对于超大规模 VRP 而

言, 这一点尤其关键, 因为许多局部连接是否合理, 往往只有放到整条路径乃至多条路径的全局协调中才能做出判断。

第二个核心困难在于, 经典启发式搜索虽强, 但在现有神经求解器中通常仍主要扮演一次性后处理器的角色。大量工作表明, 神经模型输出的边热图、候选边分数或初始解, 配合后续搜索或精炼, 往往能够显著改善最终结果 [?????]。在这一路线上, NeuroLKH 进一步说明了神经引导与经典启发式搜索结合的有效性, 即学习模块可以为 LKH 类搜索器提供有价值的候选信息, 从而提升最终解质量 [?]。然而, 在大多数方法中, 这种结合仍然是单向的。模型先预测一次, 启发式搜索再精炼一次。然而, 搜索轨迹中逐步揭示出的结构信息既不会在同一次推理中反馈给模型, 也很少被系统性地转化为后续训练监督。换言之, 现有大多数神经-启发式方法仍然更接近 open-loop pipeline, 启发式搜索能够改进当前实例, 却并未真正成为学习过程中的闭环组成部分。尽管近期已有工作开始反思将 search 仅视为 post-hoc refinement 的局限性 [????], 但如何在大规模 VRP 中把学习与经典启发式搜索后端统一为一个同时作用于推理和训练的闭环系统, 仍是一个开放问题。

本文的核心观点是, 在超大规模 VRP 中, 学习模块的职责不应是直接输出一个几乎完成的可行解, 而应是学习一个对启发式搜索友好的全局结构先验; 相应地, 经典启发式搜索也不应只是模型预测之后的一次性补救器, 而应在推理时持续反馈更优的当前结构状态, 在训练时持续反馈更强的监督信号。基于这一观点, 我们提出 **LSL** (**L**earning to **S**earch, **S**earching to **L**earn), 一个面向大规模 VRP 的神经-启发式闭环框架。与直接在解空间中恢复单条高质量路径的建模方式不同, LSL 在稀疏候选图上学习一个供下游启发式搜索消费的结构先验分布; 与将启发式搜索视为一次性后处理的做法不同, LSL 让模型预测与启发式搜索在同一实例上交替执行多轮 refinement, 并使搜索结果进一步回流为训练监督。

具体而言, LSL 由三个相互耦合的组成部分构成。首先, 我们设计了一个 *sparse diffusion transformer backbone*, 它不直接生成完整离散解, 而是在固定宽度的稀疏候选图上预测逐行结构分布, 从而在保持全局感知的同时, 将主要计算复杂度从稠密图上的 $O(N^2)$ 压缩到 $O(NK)$ 。其次, 在推理阶段, LSL 引入一个 *learning-heuristic search refinement loop*, 模型根据当前解的结构状态生成新的稀疏结构先验, 启发式搜索后端在该先验引导下继续精炼当前解, 更新后的结构状态再反馈给下一轮模型调用。这样, 经典启发式搜索不再是预测之后的单次后处理, 而是推理闭环中的显式组成部分。最后, 在训练阶段, 我们进一步引入 *search-feedback supervision loop*。对于同一个实例, LSL 保留启发式搜索轨迹中的多条高质量精炼解, 并将它们共同投影到相同的稀疏候选图上, 构造成逐行软目标分布。这样的监督目标会在被多条高质量解反复支持的局部连接上分配更高概率, 同时存在多种合理替代结构的区域保留必要的不确定性, 从而使模型学到的不是被过早锁死的单点答案, 而是更适合启发式搜索继续利用的结构先验。

这一设计带来两个直接结果。其一, 由于模型、状态表示、监督目标与启发式搜索接口始终停留在统一的稀疏候选域中, 当候选宽度 K 固定时, LSL 的主要显存占用和计算量都随实例规模近似线性增长, 使其能够在单张消费级 GPU 上直接处理最高 100K 节点的实例, 而不依赖硬分解或稠密全图建模。其二, 启发式搜索不再只服务于测试阶段的即时改进, 它还能够持续产出更优监督, 进而提升后续学习能力。更进一步地, 这种训练侧闭环还使 LSL 能够在高质量标签稀缺的超大规模场景中自举生成高质量训练数据, 例如构建可直接用于后续研究的 100K 规模高质量监督数据集, 从而为更多学习型路径优化方法提供可复用的训练资源与评测支撑。因此, LSL 的关键不只是“用启发式搜索修补模型输出”, 而是让学习与启发式搜索在推理时共同求解当前实例, 并在训练时共同塑造未来的模型行为。

我们在大规模 TSP 和 CVRP 基准上的实验表明, LSL 在解质量、推理效率与可扩展性之间取得了优异平衡, 并在多个设置下达到最先进或极具竞争力的结果。更重要的是, 这些结果表明闭环本身是有价值的, 同一个经典启发式搜索过程既能在推理时持续改善当前解, 也能在训练时持续生成更强的监督信号。总体而言, 本文的贡献可以概括为以下四个方面:

- 我们提出了 **LSL**, 一个将神经先验学习与经典启发式搜索后端统一起来的闭环框架, 用于大规模 VRP, 而不是将启发式搜索仅作为一次性 post-hoc refinement 模块。
- 我们设计了一个面向超大规模实例的实用化方案, 以稀疏候选图上的 *sparse diffusion transformer backbone* 为核心, 在尽量保留全局结构信息的同时实现关于节点规模近似线性的可扩展性。
- 我们提出了由启发式搜索轨迹诱导的逐行多解软监督, 使搜索得到的多条高质量解能够直接转化为后续训练信号, 并通过大规模 TSP/CVRP 实验证明该设计能够同时提升可扩展性、效率与解质量。

- 我们进一步表明, LSL 不仅能够提升当前求解性能, 还能够 在 100K 级别场景中自举生成高质量训练数据。这类数据可作为后续学习型路径优化研究的监督资源, 为相关方法的训练、迁移与比较提供支持。

2 学习-搜索闭环框架

2.1 VRP 问题形式化

车辆路径规划问题 (VRP) 可以统一表述为定义在全局图上的组合优化问题。给定一个实例, 记其全局图为 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, 其中 $\mathcal{V}$ 为节点集合, $\mathcal{E} = \{(i, j) \mid i, j \in \mathcal{V}, i \neq j\}$ 为全局边集。记 $\mathcal{X}(\mathcal{G})$ 为实例 $\mathcal{G}$ 的可行解空间, 并记 $\mathbf{x} \in \mathcal{X}(\mathcal{G})$ 为定义在全局图上的一个可行解。统一地, $\mathbf{x}$ 可以理解为对全局边域上结构选择关系的编码。于是, VRP 的统一优化目标可写为

$$\mathbf{x}^*(\mathcal{G}) = \arg \min_{\mathbf{x} \in \mathcal{X}(\mathcal{G})} C(\mathbf{x}; \mathcal{G}), \quad (1)$$

其中 $C(\mathbf{x}; \mathcal{G})$ 表示解 $\mathbf{x}$ 的总代价。在这一统一表述下, TSP 与 CVRP 的主要差异体现在可行域 $\mathcal{X}(\mathcal{G})$ 的约束结构上。对于 TSP, 可行解必须满足单回路约束与节点唯一访问约束; 对于 CVRP, 除客户唯一服务约束外, 还需满足 route 与 depot 的连接约束以及车辆容量约束。本文后续不再直接在全局边域上对 $\mathbf{x}$ 进行建模, 而是进一步将其映射为更适合大规模学习与搜索的结构表示; 在此之前, 我们先回顾现有基于扩散的离线解分布学习方法。

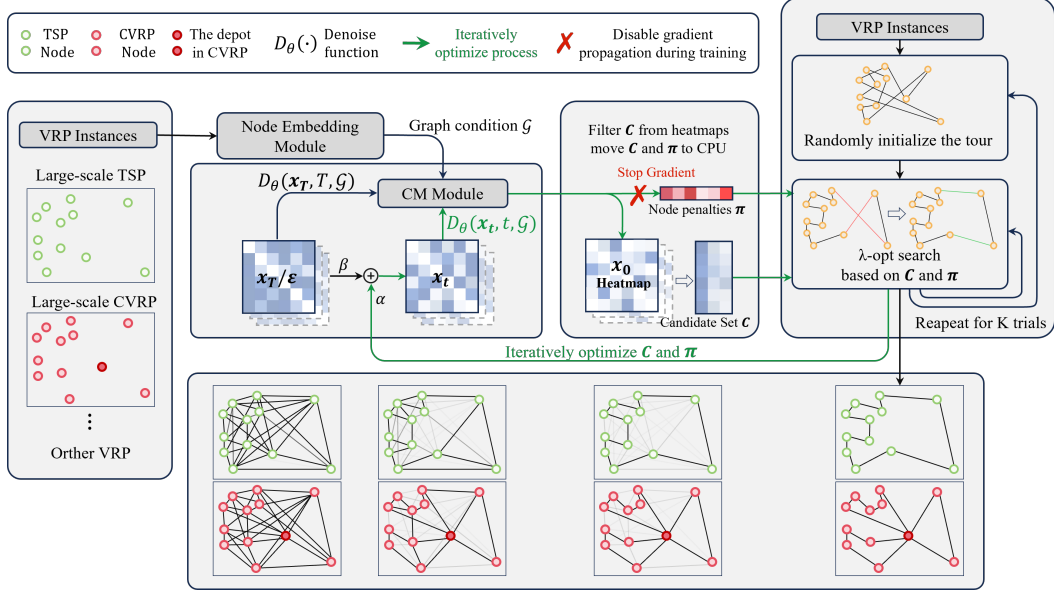


Figure 1: **LSL 总体框架**。从大规模 VRP 实例出发 (左)，节点嵌入模块先在稀疏候选图上构造实例特征。随后，稀疏扩散去噪器 D_θ 以实例图、时间步以及上一轮 search 返回的当前结构状态为条件，在同一稀疏支持上预测新的结构先验 (中左)。CM 模块负责注入 incumbent 解上下文，使模型同时利用静态几何关系与 search 已揭示出的当前结构。接着，预测热图被过滤为候选集 $\mathcal{C}$ 与节点惩罚 π (中右)；图中的“Stop Gradient”表示这一离散抽取过程及后续启发式搜索被当作非可微反馈接口使用。右侧展示了一个具体的 Guided-LKH 风格后端：它先随机初始化 tour 或 route，再在 $\mathcal{C}$ 和 π 的引导下执行多轮 λ -opt search，并在多个 trial 中保留更优精炼解。绿色回流箭头刻画了推理时的闭环关键：改进后的解会被重新投影回稀疏图，作为下一轮模型调用的结构状态，从而让 learning 与 search 在同一实例上迭代共优化。训练时，多条高质量 search 输出还会继续被投影到候选图上，形成逐行软目标。底部的 TSP/CVRP 小图展示了稀疏支持如何在保持全局有效连接的同时逐步去噪与收缩。

2.2 基于扩散的离线解分布学习

近年来，一类重要的生成式神经组合优化方法不再将求解直接视为从实例到最终解的单步映射，而是首先在离线阶段学习实例条件下的高质量解分布。给定高质量参考解 $\mathbf{x}^*$ 与一个可行的次优解 $\mathbf{x}^{\text{sub}}$ ，可以定义一条插值轨迹

$$\mathbf{x}_t = t\mathbf{x}^{\text{sub}} + (1-t)\mathbf{x}^*, t \in [0, 1] \quad (2)$$

在这种情况下，设置漂移力 $\mathbf{v}$ 以驱动流动尽可能地遵循从 $\mathbf{x}^*$ 指向 $\mathbf{x}^{\text{sub}}$ 的线性路径的方向 ($\mathbf{x}^{\text{sub}} - \mathbf{x}^*$)，即 $\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, t, G) = \mathbf{x}^{\text{sub}} - \mathbf{x}^*$ 。由于 $\mathbf{x}^*$ 对于模拟此 ODE 流而言是未知的，因此我们使用具有以下损失函数的神经网络参数化速度场 $\mathbf{v}$ ：

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}^{\text{sub}}, G} \left[\int_0^1 \|\mathbf{v}(\mathbf{x}_t, t, G) - (\mathbf{x}^{\text{sub}} - \mathbf{x}^*)\|_2^2 dt \right] \quad (3)$$

2.3 搜索友好的稀疏扩散 Transformer

现有基于扩散或 flow matching 的组合优化方法，虽然在状态空间和参数化形式上各不相同，但大多仍然围绕单个高质量参考解 $\mathbf{x}^*$ 展开学习。即使在测试阶段引入搜索，其搜索过程也主要是在模型已学习的解空间或生成轨迹内部，对当前候选解进行进一步优化。换言之，这些方法中的 search 更多扮演 test-time optimization 或 generation enhancement 的角色，而不是

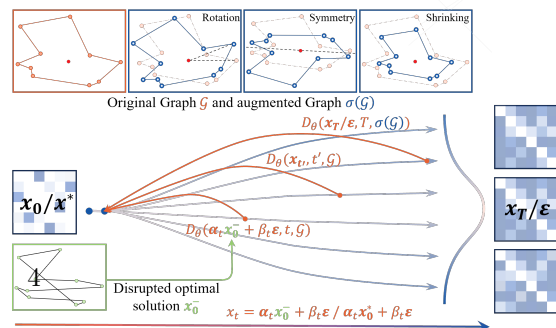


Figure 2: **推理时的结构状态构造**。稀疏结构热图先经过扰动，再通过插值得到供去噪器使用。

121 一个与模型形成显式双向反馈的外部启发
 122 式后端。对本文关注的大规模 VRP 而言，
 123 这种以单解恢复为中心的建模方式存在一个
 124 关键局限：模型学到的往往是如何逼近
 125 某一个特定高质量解，而不是如何为后续
 126 离散搜索提供真正有用的结构引导。

127 更进一步地，由于 $\mathbf{x}$ -空间中的状态通常被
 128 解释为可行解本身，或至少是朝向可行解
 129 恢复的中间变量，这类建模天然会把生成
 130 过程限制在组合优化约束附近。尤其在 CVRP 这类同时具有容量约束、depot 连接约束以及
 131 route 结构约束的问题中，这意味着模型更容易被驱动去恢复一个“接近最终可行解”的
 132 对象。这样的设计有利于直接还原某一条高质量路径或某一组高质量 routes，但未必最适合
 133 后续搜索。对于搜索而言，更有价值的往往不是一个已经被强约束到可行域附近、并且过早
 134 锁死局部决策的对象，而是一个能够表达结构偏好、却不必处处满足最终组合约束的先验分
 135 布。换言之，搜索更希望接收的是“哪里更值得搜”的信号，而不是“模型已经基本替搜索
 136 做完了”的半成品解。

137 基于这一点，我们不再将时间条件化状态仅仅视为受扰后的解变量，而是将其视为定义在图
 138 结构上的时间条件化结构先验。具体地，我们用 $\mathcal{H}$ 而不是 $\mathbf{x}$ 来表示模型所处理的结构状态，
 139 其中 $\mathcal{H}_t$ 表示在时间步 t 下的结构先验。当 t 较小时， $\mathcal{H}_t$ 与高质量结构更加接近；当 t 较大
 140 时， $\mathcal{H}_t$ 则保留更弱、更模糊或更不完整的结构信息。因此，在本文中，时间变量 t 的作用不
 141 再仅仅是控制噪声强度，而是控制当前结构先验对高质量解区域的可参考程度。模型的目标
 142 也不再是直接恢复单个高质量参考解，而是学习一个更适合后续搜索利用的 search-friendly
 143 结构先验。这里的关键点在于， $\mathcal{H}$ 本身并不被要求表示一个满足组合优化约束的可行解，它
 144 只是定义在图结构上的纯粹结构分布，因此能够为搜索保留更大的调整空间。

145 为了使这一结构先验学习在超大规模实例上真正可行，我们进一步将全局图 $\mathcal{G}$ 映射到稀疏
 146 候选图

$$\mathcal{G}_K = (\mathcal{V}, \mathcal{E}_K), \quad \mathcal{E}_K = \{(i, j) \mid j \in \mathcal{N}_K(i), |\mathcal{N}_K(i)| = K\}, \quad (4)$$

147 其中 $\mathcal{N}_K(i)$ 表示节点 i 保留的 K 个候选邻居。后续所有结构状态、模型预测与监督目标均
 148 定义在这一稀疏边域上，从而将主要计算复杂度从全局图上的 $O(N^2)$ 压缩到 $O(NK)$ 。

149 在此基础上，我们构造一个稀疏扩散 Transformer 作为 learning 模块。给定当前实例图 $\mathcal{G}_K$ 、
 150 时间步 t 以及当前结构状态 $\mathcal{H}_t$ ，模型学习映射 $D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)$ ，并输出定义在同一稀疏候选
 151 边域上的逐行结构分布 $\mathcal{H}_{\text{guide}} = D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)$ 。这里， $\mathcal{H}_{\text{guide}}$ 不被解释为最终解本身，而
 152 被解释为供后续搜索消费的结构引导信号。也就是说，这个模块的目标不是直接恢复某一条
 153 高质量 route，而是在统一的稀疏结构域上输出“哪些局部连接更值得后续搜索优先关注”
 154 的先验。

155 因此，这一稀疏扩散 Transformer 的核心作用，不是从中间状态中恢复某一个单独的高质量
 156 解，而是学习一个能够显式服务于下游启发式搜索的结构先验；而 sparse online softmax 则
 157 是这一 search-friendly 稀疏学习器在大规模场景下真正成立的重要基础。

158 **架构** 在架构上，该 learning 模块由一个稀疏编码器和一个时间条件化稀疏解码器组成。编
 159 码器在 $\mathcal{G}_K$ 的候选边域上传播节点几何信息与静态实例信息；解码器进一步将时间步 t 与
 160 当前结构状态 $\mathcal{H}_t$ 注入同一稀疏结构域中。具体而言，在解码器第 ℓ 层、第 h 个注意力头
 161 中，节点 i 指向候选节点 $j \in \mathcal{N}_K(i)$ 的注意力打分写为

$$e_{ij}^{(\ell, h)} = \frac{\langle \mathbf{q}_i^{(\ell, h)}, \mathbf{k}_j^{(\ell, h)} \rangle}{\sqrt{d_h}} + b_{ij}^{(\ell)}(\mathcal{H}_t, t), \quad (5)$$

162 其中 $\mathbf{q}_i^{(\ell, h)}$ 与 $\mathbf{k}_j^{(\ell, h)}$ 分别表示第 h 个头上的 query 和 key， d_h 为单头隐藏维度，而 $b_{ij}^{(\ell)}(\mathcal{H}_t, t)$
 163 是由当前结构状态与时间步共同诱导的加性偏置。这样，模型不仅能够感知当前实例的几何
 164 与拓扑上下文，也能够显式利用“当前已经显露出哪些结构模式”这一信息来预测更好的搜
 165 索引导。

166 为了在大规模 K 下保持数值稳定与计算效率，我们并不直接在整行候选上显式计算 softmax，
 167 而是采用按块计算的稀疏 online softmax。具体地，将候选邻居集合 $\mathcal{N}_K(i)$ 划分为若干块
 168 $\{\mathcal{B}_b(i)\}_b$ ，其中 $\mathcal{B}_b(i) \subseteq \mathcal{N}_K(i)$ 表示第 b 个候选块。记 $m_i^{(\ell, h, b)}$ 与 $z_i^{(\ell, h, b)}$ 分别为当前行在处

169 理到第 b 个块时的运行最大值与运行配分函数，则有

$$m_i^{(\ell,h,b)} = \max\left(m_i^{(\ell,h,b-1)}, \max_{j \in \mathcal{B}_b(i)} e_{ij}^{(\ell,h)}\right), \quad (6)$$

$$z_i^{(\ell,h,b)} = z_i^{(\ell,h,b-1)} \exp\left(m_i^{(\ell,h,b-1)} - m_i^{(\ell,h,b)}\right) + \sum_{j \in \mathcal{B}_b(i)} \exp\left(e_{ij}^{(\ell,h)} - m_i^{(\ell,h,b)}\right). \quad (7)$$

171 这一写法使我们无需在全候选行上显式构造稠密 softmax，就能在稀疏候选域内完成稳定归一化。对本文而言，这不仅是一个实现优化，而是稀疏扩散 Transformer 能够在超大规模实例上保持 $O(NK)$ 复杂度、同时兼顾训练稳定性与推理效率的关键组成部分。

174 在解码器末端，我们用轻量的边打分头在候选边上生成最终结构分布。记 $\mathbf{g}_i$ 为节点 i 的终层表示，则有

$$s_{ij} = \langle \mathbf{g}_i, \mathbf{g}_j \rangle, \quad \mathcal{H}_{\text{guide},ij} = \frac{\exp(s_{ij})}{\sum_{j' \in \mathcal{N}_K(i)} \exp(s_{ij'})}, \quad j \in \mathcal{N}_K(i). \quad (8)$$

176 图 ?? 总结了 LSL 的整体流程。图中右侧采用一个具体的 Guided-LKH 风格实例来展示预测热图如何被转化为离散搜索可消费的候选信息；而在后续公式中，我们把这一搜索后端统一抽象为算子 S ，以强调闭环接口本身与具体稀疏搜索实现解耦。

179 2.4 推理时的 Learning-Search 精炼闭环

180 在推理时，LSL 会在同一个实例上执行多个 learning-search cycle。记第 c 轮的当前 route set 为 $\mathcal{R}^{(c)}$ ，并记其在候选图上的投影为 $\mathcal{H}^{(c)} = P(\mathcal{R}^{(c)}; \mathcal{G}_K)$ 。对 TSP 而言，它退化为单条 tour；对 CVRP 而言，它由同一个可行解中的多条 route 组成。以这个结构状态为条件，去噪器完成一次完整的推理调用并输出引导热图 $\mathcal{H}_{\text{guide}}^{(c)}$ ，search 再在该热图引导下把预算 B 继续用于精炼当前 route set：

$$\begin{aligned} \mathcal{H}_{\text{guide}}^{(c)} &= D_\theta(\mathcal{G}_K, \mathcal{H}^{(c)}), \\ \mathcal{R}^{(c+1)} &= S(\mathcal{R}^{(c)}, \mathcal{H}_{\text{guide}}^{(c)}, B), \\ \mathcal{H}^{(c+1)} &= P(\mathcal{R}^{(c+1)}; \mathcal{G}_K). \end{aligned} \quad (9)$$

185 其中 $\mathcal{H}_{\text{guide}}^{(c)}$ 表示由参数 θ 刻画、并以实例图 $\mathcal{G}_K$ 和当前结构热图 $\mathcal{H}^{(c)}$ 为条件的稀疏扩散去噪器输出。这里我们把 $D_\theta(\mathcal{G}_K, \mathcal{H}^{(c)})$ 记作一次完整的去噪调用，为了不让公式过重，省略了它内部的时间积分记号。标量 B 表示单个 learning-search cycle 分配到的搜索预算。在我们的实现中， S 可以实例化为由热图引导的 LKH 精炼，或者稀疏 merge 式路径构造，但这个接口本身与具体后端解耦。

190 这个写法把每个推理 cycle 清楚地拆成了三步。首先，当前 route set 被压缩为候选图上的结构状态；其次，learning 把这个结构状态转成下一轮 search 要消费的引导热图；最后，search 在该引导下返回更新后的 route set。也就是说，模型并不是每一轮都只看原始实例重新开始，而是显式利用上一轮 search 刚刚发现的结构。

194 反过来，search 也不再是预测结束后的单次后处理，而是推理闭环内部的一个算子，因为它最新返回的 route set 会立即被重新投影回相同的稀疏图，并直接进入下一轮模型调用。在实现中，我们还可以在相邻 cycle 之间对 incumbent route set 施加小幅随机扰动，再进入下一轮预测。这样做是为了避免闭环反复停留在同一个局部盆地附近，而能在其邻域内探索结构上不同、但仍然有希望的解状态，同时保持 search 与 learning 始终通过同一个结构热图接口耦合。

200 2.5 训练时的搜索反馈闭环

201 在训练阶段，search 的作用不再是继续精炼当前解，而是反过来为 learning 提供更强监督。对于每一个训练实例，我们沿着 local search 轨迹保留多组高质量精炼 route set $\{\mathcal{R}^{(m)}\}_{m=1}^M$ ，并将它们重新投影到相同的稀疏候选图上，构造一个未归一化的搜索共识目标

$$\overline{\mathcal{H}}_{ij} = \sum_{m=1}^M w_m P(\mathcal{R}^{(m)}; \mathcal{G}_K)_{ij}, \quad w_m \geq 0, \quad (10)$$

其中 w_m 表示分配给 route set $\mathcal{R}^{(m)}$ 的非负权重，质量更高的解可以赋予更大的权重。接着，我们按行归一化，得到最终的软目标

$$\mathcal{H}_{ij}^* = \frac{\overline{\mathcal{H}}_{ij}}{\sum_{j' \in \mathcal{N}_K(i)} \overline{\mathcal{H}}_{ij'}}. \quad (11)$$

与单条硬标签路径不同， $\mathcal{H}^* \in \mathbb{R}_+^{N \times K}$ 提炼的是搜索轨迹中的结构共识，而不是某一个固定 teacher 解。如果多条高质量解都反复支持同一条局部连接，那么它的概率质量就会更大；如果某个局部仍存在多种合理选择，那么对应这一行就会保持较软的分布形态。

训练闭环的第二个特点是，模型并不只在“干净 teacher 状态”上训练。相反，我们在同一候选图上构造当前结构热图 $\mathcal{H}$ 与 teacher 侧结构热图 $\hat{\mathcal{H}}$ ，再用时间步对两者进行插值：

$$\mathcal{H}_t = (1 - t)\mathcal{H} + t\hat{\mathcal{H}}. \quad (12)$$

这里， $\mathcal{H}$ 来自当前 route set 的投影，而 $\hat{\mathcal{H}}$ 来自高质量 teacher 解，或者其经过 target disruption 后得到的扰动版本。这个构造的重点不是重新定义模型，而是让模型在训练时持续接触“有结构但并不完美”的中间状态，从而更贴近推理闭环里 search 实际返回的状态。

在此基础上，训练目标就是预测热图与搜索诱导软目标之间的逐行稀疏交叉熵：

$$\mathcal{L}_{\text{LSL}} = -\frac{1}{N} \sum_{i=1}^N \sum_{j \in \mathcal{N}_K(i)} \mathcal{H}_{ij}^* \log D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)_{ij}. \quad (13)$$

这里 $D_\theta(\mathcal{G}_K, \mathcal{H}_t, t)_{ij}$ 表示去噪器在实例图 $\mathcal{G}_K$ 条件下，为第 i 行中的候选边 (i, j) 分配的概率。对于 TSP，这就是完整的训练目标。对于 CVRP，我们保留同样的总体形式，只是在 Appendix ?? 中进一步加入 depot-aware 的状态构造与 depot-sensitive 的加权方式。

训练闭环的第三个特点是，一旦 search 找到了更好的解，我们就可以把它们直接转化为新的软目标，并并入下一轮训练数据：

$$\mathcal{D}^{(r+1)} = \mathcal{D}^{(r)} \cup \left\{ (\mathcal{G}_K, \mathcal{H}^{*,(r)}(\mathcal{G}_K)) \mid \mathcal{G}_K \in \mathcal{B}^{(r)} \right\}. \quad (14)$$

这里 $\mathcal{D}^{(r)}$ 表示第 r 轮之后的训练集， $\mathcal{B}^{(r)}$ 表示该轮被重新访问的一组实例图，而 $\mathcal{H}^{*,(r)}(\mathcal{G}_K)$ 表示从实例 $\mathcal{G}_K$ 最新的搜索精炼解中挖掘得到的软目标。这使得 LSL 在大规模监督稀缺的场景下，天然具有自举能力。特别是在超大规模实例上，当高质量标签难以直接获得时，框架可以利用自身的搜索精炼结果持续改写 soft target，并逐步积累更强的训练监督。

3 实验

本节从四个方面系统评估 LSL 在大规模 TSP 与 CVRP 上的表现。首先，我们在合成基准上与经典求解器、构造式 NCO、热图式 NCO 以及两阶段大规模路由方法进行比较，检验 LSL 在解质量、推理效率与可扩展性之间的整体表现。其次，我们在真实世界 TSPLIB、VLSI、Art-TSP 与 CVRPLIB 实例上评估其跨分布泛化能力。再次，我们通过闭环机制与搜索反馈监督的消融实验，验证 LSL 的核心设计是否真正带来性能增益。最后，我们进一步从推理预算、cycle 数、候选宽度以及大规模扩展曲线等角度分析该框架的行为特征。

3.1 实验设置

数据集 为系统评估 LSL 在不同问题规模下的表现，我们分别为 TSP 与 CVRP 构建了多尺度合成数据集，而不是仅在单一规模上训练和测试。具体而言，**TSP** 侧分别生成了四个规模的数据集：TSP100、TSP1K、TSP10K 与 TSP100K，其数据集大小分别为 1.28M、64K、6.4K 和 640 个实例。其中 10K 数据经过 LSL_{self-refine}，100K 数据有 LSL 自举合成。**CVRP** 侧分别生成了三个规模的数据集：CVRP100、CVRP1K 与 CVRP10K，其数据集大小分别为 1.28M、64K 和 6.4K 个实例。CVRP1K 与 CVRP10K 由 LSL 自举合成。

对于真实世界评测，我们进一步使用 TSPLIB [?]、VLSI、Art-TSP 与 CVRPLIB Set-X [?] 作为补充测试基准，以检验模型在真实分布和跨分布场景下的泛化能力。主文中主要报告合成基准上的总体结果以及真实世界基准上的聚合结果，更细粒度的逐实例结果放入附录。

除合成数据外，我们还在真实世界基准上评估模型泛化能力。TSP 侧包括 TSPLIB [?]、VLSI 与 Art-TSP；CVRP 侧包括 CVRPLIB Set-X 和 Set-SL [?]。主文中报告按数据集与规模区间聚合后的结果，更细粒度的逐实例结果见附录。

244 **基线方法** 我们将 LSL 与以下方法进行比较: (1) **经典求解器**: Concorde [?]、LKH3 [?]]
245 和 HGS [?]; (2) **构造式 NCO**: POMO [?]、BQ [?]、LEHD [?]、ELG [?]、INViT [?]
246 以及 L2C-Insert [?]; (3) **热图式或生成式 NCO**: Att-GCN+MCTS [?]、DIFUSCO [?]、
247 T2T [?]、Fast-T2T [?] 和 GenSCO [?]; (4) **两阶段或大规模方法**: H-TSP [?]、GLOP [?]
248 和 UDC [?]。对于部分仅公开论文结果、难以在统一环境下完全复现的基线, 我们沿用
249 原论文报告值, 并在表格中以星号标注。

Table 1: 合成 TSP 和 CVRP 实例上的实验结果。标有星号 (*) 的结果来自原论文。最优结果加粗显示。OOM: 方法触发内存溢出。所报告的推理时间为求解整个测试集所需的总时间。

Method	TSP100			TSP1K			TSP10K		
	Length	Gap	Time	Length	Gap	Time	Length	Gap	Time
LKH3	7.76325	0.000%	56m	23.119	0.000%	8.2h	71.778	0.000%	8.8h
Concorde	7.76324	0.000%	34m	23.118	-0.003%	7.8h	72	0.310%	22.4h
Att-GCN+MCTS*	7.764	0.037%	15m	23.863	3.224%	13m	74.93	4.390%	1.8h
DIFUSCO*	7.78	0.240%	-	23.56	1.900%	12.1m	73.62	2.580%	47m
T2T*	7.76	0.060%	-	23.3	0.780%	54.7m	-	-	-
Fast-T2T*	7.76	0.010%	-	23.22	0.420%	18.3m	72.94	1.630%	-
GenSCO*	7.76	0.000%	-	23.13	0.036%	3.9m	-	-	-
H-TSP	-	-	-	24.718	6.912%	34s	77.75	8.320%	25m
GLOP	7.767	0.048%	1.4h	23.84	3.119%	2.4m	75.04	4.545%	25m
UDC- $\mathbf{x}_{50}$ ($\alpha = 50$)	7.788	0.319%	1.38h	23.53	1.782%	6.21m	OOM	OOM	OOM
POMO aug $\times 8$	7.774	0.138%	0.7m	32.5	40.577%	6.1m	OOM	OOM	OOM
ELG	7.781	0.229%	2.2m	25.738	11.328%	1.3m	OOM	OOM	OOM
BQ bs16	7.764	0.010%	20m	23.432	1.354%	21m	OOM	OOM	OOM
LEHD RRC1000	7.763	0.002%	2.3h	23.288	0.731%	5.2h	80.9	12.709%	3.6h
L2C-Insert Greedy	7.799	0.459%	0.86m	24.218	4.754%	0.35m	77.343	7.754%	1.06m
L2C-Insert ($I=1000$)	7.763	0.000%	8.0h	23.230	0.481%	46.4m	73.270	2.079%	16.64m
LSL ($C=1, B=128$)	7.763	0.000%	4s	23.119	0.000%	1.7m	71.800	0.030%	1.2m
LSL ($C=4, B=512$)	-	-	-	23.118	-0.002%	15.5m	71.775	-0.005%	16.9m

Method	CVRP100			CVRP1K			CVRP10K		
	Length	Gap	Time	Length	Gap	Time	Length	Gap	Time
HGS	15.56	0.000%	4.5h	41.161	0.000%	5h	226.177	0.000%	8h
LKH3	15.65	0.540%	12h	42.156	2.416%	6.1h	290.575	28.472%	10.3h
GLOP-G (LKH3)	-	-	-	45.753	11.154%	1.93m	276.252	22.140%	27.98m
UDC- $\mathbf{x}_{50}$ ($\alpha = 50$)	16.291	4.699%	2.69h	43.583	5.884%	11.96m	OOM	OOM	OOM
POMO aug $\times 8$	15.75	1.230%	0.86m	100.990	>100%	5.83m	OOM	OOM	OOM
ELG	15.84	1.760%	3.4m	46.847	13.812%	3.16m	277.685	22.773%	11.92m
BQ bs16	15.81	1.560%	18m	43.121	4.762%	15m	275.494	21.805%	1.36h
LEHD RRC1000	15.63	0.420%	2.0h	42.408	3.028%	4.15h	248.604	9.916%	8.1h
L2C-Insert Greedy	16.169	3.892%	1.4m	44.458	8.009%	1.17m	255.356	12.901%	2.88m
L2C-Insert ($I=1000$)	15.627	0.413%	10.6h	43.152	4.836%	2.05h	245.732	8.646%	34.72m
LSL ($C=1, B=128$)									
LSL ($C=4, B=512$)									

推理协议与评价指标 图 ?? 展示了输入去噪器的中间结构热图如何构造。

推理时, LSL 首先在稀疏候选图上预测结构先验, 再由启发式后端在固定预算下进行离散精炼。除特别说明外, 默认设置使用 aug $\times 8$, 并报告两个代表性配置: 轻量配置 LSL ($C=1, B=128$) 与更强配置 LSL ($C=4, B=512$), 其中 C 表示 learning-search cycle 数, B 表示每个 cycle 的搜索预算。我们报告三类指标: (1) 平均路径长度/总代价 (Length), (2) 相对参考求解器的性能差距 (Gap), (3) 求解整个测试集的总 wall-clock 时间 (Time)。需要说明的是, 对于来自原论文的时间结果, 我们主要将其作为参考, 不将其与本地统一硬件下获得的时间结果作过强的绝对速度比较; 更细粒度的单实例平均推理时间见附录。

3.2 合成基准上的主结果

表 ?? 和表 ?? 给出了合成 TSP 与 CVRP 基准上的主结果。整体上, LSL 在不同规模下均取得了稳定且有竞争力的表现。对于 TSP, LSL 在 TSP100 和 TSP1K 上已达到与强经典启发式接近的结果; 当规模增大到 10K、50K 和 100K 时, LSL 仍保持极低 gap, 而多种构造式或稠密热图式方法则出现明显退化甚至 OOM。尤其在大规模设置下, LSL 相比代表性的神经基线和两阶段方法 consistently 取得更优的质量-时间权衡, 说明其稀疏结构先验与启发式精炼的组合更适合超大规模实例。

对于 CVRP, LSL 同样表现出良好的扩展性。在 CVRP100、CVRP1K 和 CVRP10K 上, LSL 持续优于大多数神经基线, 并在解质量与推理时间之间取得更优平衡。相比直接构造 routes 的方法, LSL 不要求模型独立完成全部离散可行性决策, 而是通过 depot-aware 的稀疏结构引导与后续启发式 refinement 协同求解, 因此在规模增大时更稳定。

Table 3: TSPLIB、VLSI、Art-TSP 与 CVRPLIB 实例上的对比结果。

Method	TSPLIB		VLSI		Art-TSP		CVRPLIB	
	≤ 1000	> 1000	≤ 1000	> 1000	≤ 1000	> 1000	≤ 1000	> 1000
UDC- x_{250} ($\alpha = 50$)	3.072%		-		-		9.033%	
GLOP	1.153%		-		-		-	
ELG	3.111%		-		-		7.039%	
DIFUSCO	1.736%		-		-		-	
Fast T2T	0.542%		-		-		-	
GenSCO	0.184%		-		-		-	
BQ bs16	1.557%		-		-		8.078%	
L2C-Insert ($I=1000$)	0.308%		2.915%		-		3.201%	
LSL ($C=1, B=128$)								
LSL ($C=4, B=512$)	0.003%		-		-		-	

Table 4: 推理时闭环机制的消融实验。在相同或相近总预算下比较不同 learning-search 组合方式。

Method	Learned guide	Guide refreshed	Search	TSP1K			TSP10K		
				Length	Gap	Time	Length	Gap	Time
Search-only(No guide)			✓						
One-shot Guide + 单解硬标签 guide	✓		✓						
One-shot Guide + 多解软目标 guide	✓		✓						
LSL Full ($C=4$)	✓	✓	✓						

从 TSP 侧结果看, LSL 在中大规模基准上能够稳定逼近甚至超过强经典启发式参考, 并显著优于多类现有神经方法。尤其在 10K、50K 与 100K 节点设置下, 许多构造式或稠密热图式方法已出现明显性能退化或直接 OOM, 而 LSL 仍能在单张消费级 GPU 上保持可用的推理时间和较低 gap。这表明, 以稀疏结构先验替代直接恢复完整离散解, 有助于在超大规模场景中兼顾全局感知与可扩展性。

从 CVRP 侧结果看, LSL 同样保持出色的可扩展表现。相较于直接构造 routes 的方法, LSL 并不要求模型在前向过程中独自解决全部离散可行性约束, 而是通过 depot-aware 的稀疏状态表示与后续启发式精炼分担决策负担。因此, 随着规模增大, LSL 往往能获得更好的稳定性与更优的质量-时间折中。

3.3 真实世界基准上的结果

表 ?? 进一步汇总了真实世界 TSPLIB、VLSI、Art-TSP 与 CVRPLIB 上的结果。尽管这些实例的分布与合成训练数据存在差异, LSL 仍保持了很强的泛化能力。逐实例结果见附录。

3.4 Learning-Search 闭环是否真正有效?

LSL 的核心主张之一, 是启发式搜索不应仅被视作预测结束后的单次后处理, 而应作为推理闭环内部的显式组成部分。为验证这一点, 我们在相同搜索后端与接近预算下比较以下几种设置: (1) **Search-only**, 不使用学习先验, 仅运行启发式搜索; (2) **One-shot Guide + 单解硬标签**, 仅在初始阶段预测一次结构先验, 随后固定该先验执行搜索, 并使用单条高质量解构造 one-hot 监督; (3) **One-shot Guide + 多解软目标**, 仅在初始阶段预测一次结构先验, 随后固定该先验执行搜索, 并使用多组高质量解共同构造行级 soft target; (4) **LSL Full**, 即本文所提出的 closed-loop 版本, 每轮都根据当前结构状态重新预测 guide 并反馈给搜索器。

表 ?? 用于量化不同推理范式之间的差异。我们预期, 当总预算固定时, LSL Full 能持续优于 one-shot 或 static 版本; 这将直接说明性能提升并不只是来自“再多跑几轮搜索”, 而是来自 learning 与 search 在同一实例上的显式双向耦合。为进一步展示这种优势, 我们还建议在正文或附录中给出一张 anytime curve, 画出 gap 随 wall-clock time 或 cycle index 的变化趋势。

Table 2: 含 50K/100K 节点的合成 TSP 实例上的实验结果。最优结果加粗显示。

Method	TSP50K			TSP100K		
	Length	Gap	Time	Length	Gap	Time
LKH3	159.93	0.000%	160h	225.99	0.000%	400h
GLOP	168.09	5.102%	24m	237.61	5.142%	1.04h
INViT greedy	171.42	7.184%	20.8h	242.26	7.199%	80h
LEHD RRC1000	OOM	OOM	OOM	OOM	OOM	OOM
L2C-Insert Greedy	171.778	7.408%	5.28m	242.757	7.419%	10.53m
L2C-Insert ($I=1000$)	166.063	3.835%	20.83m	237.109	4.920%	26.07m
LSL ($C=1, B=128$)						
LSL ($C=4, B=512$)						
LSL ($C=8, B=1024$)						

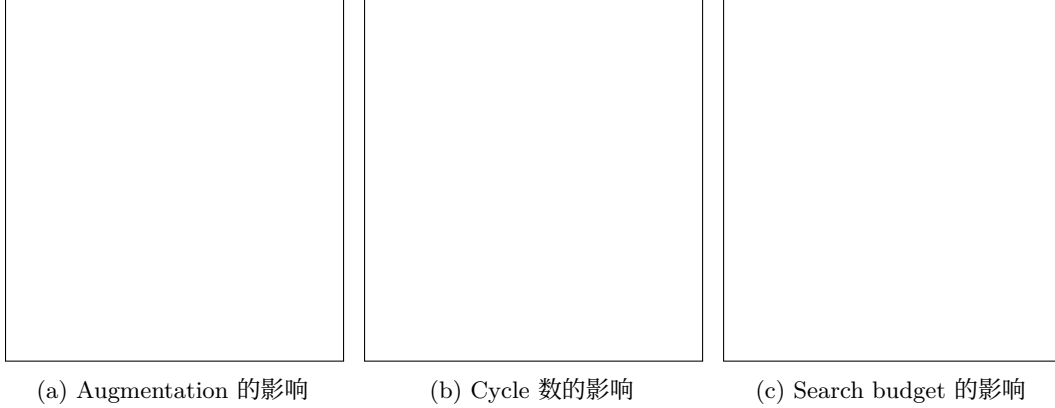


Figure 3: Augmentation、cycle 数与 search budget 对 LSL 性能的影响。

3.5 更多消融实验

除了闭环机制与监督形式之外，我们还从以下几个方面分析 LSL 的关键设计。

LSL 是否真的帮助启发式求解器跳出局部最优？ 这一问题的核心并不是比较最终终点，而是比较在相同或相近预算下，LSL 所提供的 guide 是否能让搜索轨迹进入更优的 basin。我们建议给出两类证据：其一，统计在固定 budget 下，LSL-guide 相比 search-only 的平均 improvement；其二，通过人工构造远距离的陷入局部最优的数据集，对比 search-only 和 closed-loop 跳出局部最优的平均时间。

Augmentation、cycle、搜索预算的影响 由于 LSL 的推理与训练都具有明确的预算控制维度，因此需要进一步检验 $\text{aug} \times n$ 、cycle 数 C 、target disruption 强度等因素的作用。一般而言，augmentation 与 cycle 的增加应带来更好的解质量，但收益会逐渐减弱；target disruption 则主要通过减小训练状态与推理状态之间的分布差距来提升泛化。相关结果可由图 ?? 汇报。

4 结论、局限性与未来工作

结论 本文提出了 LSL，一个面向大规模 VRP 的学习—搜索闭环框架。通过将保留全局信息的稀疏生成骨干、启发式精炼后端以及多解搜索反馈结合起来，LSL 能够生成高质量搜索先验、高效改进解，并持续提升其自身监督信号。与先进神经求解器的大量对比实验表明，LSL 在 TSP 和 CVRP 上均取得了很强性能。

局限性与未来工作 首先，尽管 LSL 已经具备良好可扩展性，但稀疏骨干与搜索后端之间仍可以做得更紧密，从而让学习到的先验与下游局部算子形成更强协同。其次，当前框架使用的是固定族的启发式精炼算子；学习搜索控制策略或自适应预算分配，是一个很有前景的下一步。第三，虽然当前的多解软目标已经保留了有价值的不确定性，但如果对搜索得到的不同解引入更丰富的加权机制，反馈环质量可能还会进一步提升。最后，将 LSL 扩展到更广泛的约束组合优化问题上，也是未来的重要方向。

References

- Bruce L Golden, S Raghavan, and Edward A Wasil. *The Vehicle Routing Problem: Latest Advances and New Challenges*, volume 43. Springer Science & Business Media, 2008.
- Paolo Toth and Daniele Vigo. *Vehicle routing: problems, methods, and applications*. SIAM, 2014.
- Victor Pillac, Michel Gendreau, Christelle Gu  ret, and Andr  s L Medaglia. A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1):1–11, 2013.

- 330 [1] Grigorios D Konstantakopoulos, Sotiris P Gayialis, and Evripidis P Kechagias. Vehicle routing
331 problem and related algorithms for logistics distribution: A literature review and classification.
332 *Operational research*, 22(3):2033–2062, 2022.
- 333 [2] Giorgio Ausiello, Pierluigi Crescenzi, Giorgio Gambosi, Viggo Kann, Alberto Marchetti-
334 Spaccamela, and Marco Protasi. *Complexity and approximation: Combinatorial optimization*
335 *problems and their approximability properties*. Springer Science & Business Media, 2012.
- 336 [3] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling
337 salesman and vehicle routing problems. *Roskilde: Roskilde University*, 12, 2017.
- 338 [4] Thibaut Vidal. Hybrid genetic search for the cvrp: Open-source implementation and swap*
339 neighborhood. *Computers & Operations Research*, 140:105643, 2022.
- 340 [5] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial
341 optimization: a methodological tour d’ horizon. *European Journal of Operational Research*,
342 290(2):405–421, 2021.
- 343 [6] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural*
344 *Information Processing Systems*, volume 28, 2015.
- 345 [7] Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi, and Samy Bengio. Neural combi-
346 natorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.
- 347 [8] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems!
348 In *International Conference on Learning Representations*, 2019.
- 349 [9] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai
350 Min. Pomo: Policy optimization with multiple optima for reinforcement learning. In *Advances*
351 *in Neural Information Processing Systems*, volume 33, pages 21188–21198, 2020.
- 352 [10] Fu Luo, Xi Lin, Fei Liu, Qingfu Zhang, and Zhenkun Wang. Neural combinatorial optimization
353 with heavy decoder: Toward large scale generalization. In *Advances in Neural Information*
354 *Processing Systems*, 2023.
- 355 [11] Darko Drakulic, Sofia Michel, Florian Mai, Arnaud Sors, and Jean-Marc Andreoli. BQ-NCO:
356 Bisimulation quotienting for efficient neural combinatorial optimization. In *Advances in Neural*
357 *Information Processing Systems*, 2023.
- 358 [12] Yan Jin, Yuandong Ding, Xuanhao Pan, Kun He, Li Zhao, Tao Qin, Lei Song, and Jiang Bian.
359 Pointerformer: deep reinforced multi-pointer transformer for the traveling salesman problem.
360 In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- 361 [13] Jianan Zhou, Yaoxin Wu, Wen Song, Zhiguang Cao, and Jie Zhang. Towards omni-
362 generalizable neural methods for vehicle routing problems. In *International Conference on*
363 *Machine Learning*, pages 42769–42789. PMLR, 2023.
- 364 [14] Yang Wang, Ya-Hui Jia, Wei-Neng Chen, and Yi Mei. Distance-aware attention reshaping:
365 Enhance generalization of neural solver for large-scale vehicle routing problems. *arXiv preprint*
366 *arXiv:2401.06979*, 2024.
- 367 [15] Changliang Zhou, Xi Lin, Zhenkun Wang, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang.
368 Instance-conditioned adaptation for large-scale generalization of neural combinatorial optimiza-
369 tion. *arXiv preprint arXiv:2405.01906*, 2024.
- 370 [16] Fu Luo, Xi Lin, Yaoxin Wu, Zhenkun Wang, Tong Xialiang, Mingxuan Yuan, and Qingfu
371 Zhang. Boosting neural combinatorial optimization for large-scale vehicle routing problems.
372 In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=TbTJJNjumY>.
373
- 374 [17] Yuanyao Chen, Rongsheng Chen, Fu Luo, and Zhenkun Wang. Improving generalization
375 of neural combinatorial optimization for vehicle routing problems via test-time projection
376 learning. In *Advances in Neural Information Processing Systems*, 2025.
- 377 [18] Fu Luo, Xi Lin, Mengyuan Zhong, Fei Liu, Zhenkun Wang, Jianyong Sun, and Qingfu Zhang.
378 Learning to insert for constructive neural vehicle routing solver. In *Advances in Neural Infor-*
379 *mation Processing Systems*, 2025.

- 380 [] Xuanhao Pan, Yan Jin, Yuandong Ding, Mingxiao Feng, Li Zhao, Lei Song, and Jiang Bian.
381 H-tsp: Hierarchically solving the large-scale traveling salesman problem. In *Proceedings of the*
382 *AAAI Conference on Artificial Intelligence*, February 2023.
- 383 [] Hanni Cheng, Haosi Zheng, Ya Cong, Weihao Jiang, and Shiliang Pu. Select and optimize:
384 Learning to solve large-scale tsp instances. In *International Conference on Artificial Intelli-*
385 *gence and Statistics*, pages 1219–1231. PMLR, 2023.
- 386 [] Qingchun Hou, Jingwei Yang, Yiqiang Su, Xiaoqing Wang, and Yuming Deng. Generalize
387 learned heuristics to solve large-scale vehicle routing problems in real-time. In *International*
388 *Conference on Learning Representations*, 2023.
- 389 [] Haoran Ye, Jiarui Wang, Helan Liang, Zhiguang Cao, Yong Li, and Fanzhang Li. Glop:
390 Learning global partition and local construction for solving large-scale routing problems in
391 real-time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- 392 [] Zhi Zheng, Changliang Zhou, Tong Xialiang, Mingxuan Yuan, and Zhenkun Wang. UDC: A
393 unified neural divide-and-conquer framework for large-scale combinatorial optimization prob-
394 lems. In *Advances in Neural Information Processing Systems*, 2024.
- 395 [] Chengrui Gao, Haopu Shang, Ke Xue, Dong Li, and Chao Qian. Towards generalizable neural
396 solvers for vehicle routing problems via ensemble with transferrable local policy. In *Interna-*
397 *tional Joint Conference on Artificial Intelligence*, 2024.
- 398 [] Han Fang, Zhihao Song, Paul Weng, and Yutong Ban. INVit: A generalizable routing prob-
399 lem solver with invariant nested view transformer. In *International Conference on Machine*
400 *Learning*, 2024.
- 401 [] Chaitanya K Joshi, Thomas Laurent, and Xavier Bresson. An efficient graph convolutional
402 network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227*, 2019.
- 403 [] Zhang-Hua Fu, Kai-Bin Qiu, and Hongyuan Zha. Generalize a small pre-trained model to
404 arbitrarily large tsp instances. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
405 volume 35, pages 7474–7482, 2021.
- 406 [] Ruizhong Qiu, Zhiqing Sun, and Yiming Yang. DIMES: A differentiable meta solver for
407 combinatorial optimization problems. In *Advances in Neural Information Processing Systems*,
408 2022.
- 409 [] Zhiqing Sun and Yiming Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial
410 optimization. In *Advances in Neural Information Processing Systems*, 2023.
- 411 [] Yang Li, Jinpei Guo, Runzhong Wang, and Junchi Yan. From distribution learning in training
412 to gradient search in testing for combinatorial optimization. In *Advances in Neural Information*
413 *Processing Systems*, 2023.
- 414 [] Yang Li, Jinpei Guo, Runzhong Wang, Hongyuan Zha, and Junchi Yan. Fast t2t: Op-
415 timization consistency speeds up diffusion-based training-to-testing solving for combinato-
416 rial optimization. In *Advances in Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=xDrKZOZE0c>.
- 417 [] Yang Li, Lvda Chen, Haonan Wang, Runzhong Wang, and Junchi Yan. Generation as search
418 operator for test-time scaling of diffusion-based combinatorial optimization. In *Advances in*
419 *Neural Information Processing Systems*, 2025. URL [https://github.com/Thinklab-SJTU/](https://github.com/Thinklab-SJTU/GenSCO)
420 [GenSCO](https://github.com/Thinklab-SJTU/GenSCO).
- 421 [] Yining Ma, Zhiguang Cao, and Yeow Meng Chee. Learning to search feasible and infeasible
422 regions of routing problems with flexible neural k-opt. In *Advances in Neural Information*
423 *Processing Systems*, volume 36, 2023.
- 424 [] André Hottung, Paula Wong-Chung, and Kevin Tierney. Neural deconstruction search for
425 vehicle routing problems. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.
426 URL <https://openreview.net/forum?id=bCmEP1Ltwq>.
- 427 [] Liang Xin, Wen Song, Zhiguang Cao, and Jie Zhang. Neurolkh: Combining deep learning
428 model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. In
429 *Advances in Neural Information Processing Systems*, volume 34, pages 7472–7483, 2021.
- 430

- 431 [] Yifan Xia, Xianliang Yang, Zichuan Liu, Zhihao Liu, Lei Song, and Jiang Bian. Position:
432 Rethinking post-hoc search-based neural approaches for solving large-scale traveling salesman
433 problems. In *International Conference on Machine Learning*, 2024. URL [https://openreview.](https://openreview.net/forum?id=cEJ9jNJJP)
434 [net/forum?id=cEJ9jNJJP](https://openreview.net/forum?id=cEJ9jNJJP).
- 435 [] André Hottung, Yeong-Dae Kwon, and Kevin Tierney. Efficient active search for combinatorial
436 optimization problems. In *International Conference on Learning Representations*, 2022.
- 437 [] Fu Luo, Xi Lin, Zhenkun Wang, Tong Xialiang, Mingxuan Yuan, and Qingfu Zhang.
438 Self-improved learning for scalable neural combinatorial optimization. *arXiv preprint*
439 *arXiv:2403.19561*, 2024.
- 440 [] Jonathan Pirnay and Dominik G. Grimm. Self-improvement for neural combinatorial opti-
441 mization: Sample without replacement, but improvement. *Transactions on Machine Learning*
442 *Research*, 2024. ISSN 2835-8856. Featured Certification.
- 443 [] Gerhard Reinelt. TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*,
444 3(4):376–384, 1991.
- 445 [] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, and Anand Sub-
446 ramanian. New benchmark instances for the capacitated vehicle routing problem. *European*
447 *Journal of Operational Research*, 257(3):845–858, 2017.
- 448 [] David Applegate, Ribert Bixby, Vasek Chvatal, and William Cook. Concorde tsp solver, 2006.
- 449 [] Thibaut Vidal. Hybrid genetic search for the cvrp: Open-source implementation and swap*
450 neighborhood. *Computers & Operations Research*, 140:105643, 2022.
- 451 [] André Hottung, Mridul Mahajan, and Kevin Tierney. Polynet: Learning diverse solution
452 strategies for neural combinatorial optimization. In *International Conference on Learning*
453 *Representations*, 2025. URL <https://openreview.net/forum?id=TKuYWeFE6S>.
- 454 [] Yong Liang Goh, Zhiguang Cao, Yining Ma, Jianan Zhou, Mohammed Haroon Dupty, and
455 Wee Sun Lee. Shield: Multi-task multi-distribution vehicle routing solver with sparsity and
456 hierarchy. In *International Conference on Machine Learning*, 2025.
- 457 [] Dian Meng, Zhiguang Cao, Jie Gao, Yaixin Wu, and Yaqing Hou. Uniteformer: Unifying
458 node and edge modalities in transformers for vehicle routing problems. In *Advances in Neural*
459 *Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=BRklmF1CsD>.
- 460 [] Changliang Zhou, Xi Lin, Zhenkun Wang, and Qingfu Zhang. L2r: Learning to reduce search
461 space for generalizable neural routing solver, 2025. URL <https://arxiv.org/abs/2503.03137>.
- 462 [] Zhang-Hua Fu, Sipeng Sun, Jintong Ren, Tianshu Yu, Haoyu Zhang, Yuanyuan Liu, Lingxiao
463 Huang, Xiang Yan, and Pinyan Lu. A hierarchical destroy and repair approach for solving
464 very large-scale travelling salesman problem. *arXiv preprint arXiv:2308.04639*, 2023.
- 465 [] Éric D Taillard and Keld Helsgaun. Popmusic for the travelling salesman problem. *European*
466 *Journal of Operational Research*, 272(2):420–429, 2019.

A 相关工作

A.1 VRP 的构造式与非构造式 NCO

面向 VRP 的神经组合优化 (NCO) 大体可以分为构造式与非构造式两类。构造式方法按序逐步生成一条路线或一组路线。Pointer Networks [?] 奠定了经典的自回归求解范式, 随后基于注意力的求解器在训练稳定性和解质量上显著推进了这一方向 [? ?]。近年来, 这一分支又通过更强的解码器、对称性感知训练和更灵活的构造动作持续演进, 包括 BQ-NCO [?], Heavy Decoder / LEHD [?], PolyNet [?], 以及最新的插入式构造模型 L2C-Insert [?]. 这类方法的共同优势在于能够学习从实例到可行解的直接映射; 但随着问题规模增大, 纯粹的序列式构造仍然更难维持长程协同、修正早期决策错误, 并在跨区域或跨路线层面保持全局一致性。

非构造式方法则更倾向于先预测中间结构, 或围绕候选解进行迭代改进。早期的热图式或结构打分方法, 如 Efficient GNN、Att-GCN+MCTS 和 DIMES, 通常先学习边概率或结构得分, 再通过束搜索、MCTS 或梯度细化恢复最终路线 [? ? ?]。更近一批生成式方法则把这一方向明显推进。DIFUSCO 将图扩散模型系统性引入组合优化, 并使扩散式边预测成为有竞争力的神经骨干 [?]; T2T 进一步把这一路线从单纯的分布学习推进到测试阶段的目标引导梯度搜索 [?]; Fast T2T 通过 optimization consistency 显著压缩该范式的推理代价, 使少步生成与高效测试时细化成为可能 [?]; GenSCO 则进一步把 generation 本身重塑为 search operator, 通过“解扰动 + 生成增强”的交替过程获得更稳定的 inference-time scaling 效果 [?]. 这条演化链说明, 非构造式 NCO 已不再局限于 one-shot 热图预测, 而是越来越强调学习可在测试时被主动搜索与细化的解分布。

与上述路线相关但不完全相同的一支工作, 则学习改进算子或把神经引导注入强搜索器。NeuOpt 和 Neural Deconstruction Search 学习局部修复或破坏-修复式 refinement 策略 [? ?], NeuroLKH 则将学习到的候选引导信息注入 Lin-Kernighan-Helsgaun 这一强经典搜索器中 [?]. 相较于纯构造式求解器, 这些方法通常拥有更强的推理时纠错能力, 但也更依赖下游搜索过程本身。因此, 在大规模 VRP 上, 核心张力依然存在: 如何在可扩展计算、全局建模与有效搜索反馈之间取得平衡, 而不让某一个模块主导整个求解流程。

A.2 面向大规模 VRP 的可扩展学习方法

如何将学习型 VRP 求解器从小中规模扩展到更大实例, 已经成为该领域的核心问题之一。近年的一个明显趋势, 是不再只依赖硬分解, 而是通过跨尺度泛化与测试时适配来直接提升大规模能力。代表性的最新工作包括 Boosting Neural Combinatorial Optimization for Large-Scale Vehicle Routing Problems, 它通过轻量线性复杂度骨干与 self-improved training 推动构造式求解器直接扩展到 100K 节点规模 [?]; Test-Time Projection Learning, 它通过学习测试阶段的投影策略来对齐大规模子图与训练分布, 而无需重新训练基础模型 [?]; 以及 L2C-Insert, 它表明更强的构造动作空间, 即从 append-only 解码转向 insertion-based construction, 也可以带来从 100 到 100K 节点的强扩展表现 [?]. 围绕这些 2025 年代表作, 更早且互补的工作如 Pointerformer、omni-generalizable routing solver、distance-aware attention reshaping、instance-conditioned adaptation、可迁移局部策略集成和 INViT, 也在逐步改善跨尺度或跨分布迁移所需的归纳偏置 [? ? ? ? ? ?]。

另一条较新的路线, 则从更高效的生成骨干或搜索辅助骨干出发追求大规模求解。DIFUSCO、T2T、Fast T2T 和 GenSCO 表明, 扩散式求解器并不一定要停留在固定的多步去噪 pipeline 上, 而可以被重新设计为更少生成步数、更强测试时搜索能力以及更高 inference-time scaling 效率的求解骨干 [? ? ? ?]。尽管这些工作大多首先在 TSP 或更广义的 CO 基准上展开, 而非完整覆盖各类 VRP 变体, 但它们对大规模路由依然高度相关, 因为它们把关注点从“如何提升单次前向预测质量”转向了“如何提升测试时计算预算的性价比”。

第三条路线则通过层次化、分区或显式局部结构来获得可扩展性。H-TSP、GLOP 和 UDC 等方法将超大实例拆解为可处理的子问题、修正阶段或 reviser 流程 [? ? ?]; 而 SHIELD 和 UniteFormer 等更新的统一化方法, 则把讨论进一步拓展到 multi-task、multi-distribution 乃至 multi-modal generalization [? ?]。这些方法显著扩大了神经路由方法的适用范围, 但其中不少方法也是通过显式限制 locality、partition 结构或决策表示来获得可扩展性或鲁棒性。对于超大规模 VRP, 这一点尤为关键, 因为真正困难的决策往往依赖远距离、跨区域的结构耦合, 而不只是局部子问题本身。

A.3 混合式学习 + 搜索方法

混合式方法将学习模型与显式搜索、启发式规则或经典求解器结合起来。常见模式是先由神经网络提供评分、候选边或候选动作，再由搜索过程生成最终解，例如基于束搜索的解码 [?]、MCTS 风格推理 [?]、扩散模型结合局部搜索 [?]、training-to-testing 梯度搜索及其加速版本 [?]、GenSCO 式的 search-driven generation [?]，以及为 LKH 提供神经引导的 NeuroLKH [?]。类似地，NeuOpt 和 Neural Deconstruction Search 也说明了学习策略能够有效驱动局部修复或破坏-修复过程 [?]。

这类方法通常比纯端到端预测取得更强的最终性能，但在多数工作中，搜索主要仍扮演后处理、解码器或单次改进模块的角色。近期也有研究明确反思，仅依赖 post-hoc search 是否足以支撑大规模泛化 [?]。换言之，许多学习 + 搜索方法仍然是开放环（open-loop）的：搜索可以帮助解好当前实例，但搜索过程中暴露出的结构信息并没有被系统性地回流到后续学习中。因此，如何把搜索从“外接求解器”转变为持续产出学习信号的组成部分，仍然是一个开放问题。

A.4 自我提升、适配与闭环邻近方法

近年来，一些工作开始从静态神经求解器转向具有适配或自我提升能力的流程。在这一方向上，[?] 表明，更强的解或重标注数据可以被反馈回训练过程，从而逐步提升可扩展路由求解器的表现。Instance-conditioned adaptation 关注预训练模型如何根据当前输入分布调整自身行为 [?]，L2R 则通过学习缩减搜索空间，把结构性引导蒸馏为后续求解可利用的先验 [?]。这些方向的重要意义在于，它们都不再把模型视为一次训练后固定不变的预测器，而是开始承认优秀路由策略应当随着所发现的解不断演化。

不过，现有自我提升或适配方法通常只覆盖闭环中的一部分。一层含义是，搜索或适配过程能够提供结构反馈、更好的先验，或为后续推理收缩更聚焦的决策空间；另一层含义是，搜索还能发现优于原始监督的数据，从而支持迭代式自我精炼，而不是始终只依赖初始数据集学习。多数现有方法触及了其中一层，但尚未充分把两层统一为一个完整的“Searching to Learn”过程，即让搜索既改变模型对结构的理解方式，又持续为后续轮次产生更优监督。

总体而言，已有工作已经从构造式/非构造式求解、可扩展学习、混合式搜索以及自我提升/适配等多个方向，为神经 VRP 求解打下了坚实基础。LSL 与这些路线并非对立，而是在它们的交汇处进一步前移：相较于通过硬分解、局部限制或阶段化求解来换取规模能力的方法，LSL 更强调在超大规模 VRP 中尽量保留全局感知；相较于把搜索视作 post-hoc 解码器或单次 refinement 模块的方法，LSL 更强调让搜索持续产出结构反馈和更优监督；相较于仅覆盖适配、重标注或自提升闭环单一环节的方法，LSL 试图把“Learning to Search”和“Searching to Learn”统一为同一个可迭代的系统框架。

550 B Network architecture details

551 B.1 统一问题形式

552 我们用同一个 $n \times K$ row-slot 稀疏图来描述 TSP 和 CVRP，而不是在关键路径上显式构造
553 $n \times n$ 稠密图。对任意节点 i ，我们只保留 K 个候选槽位

$$\mathcal{I}_i = (j_{i,0}, j_{i,1}, \dots, j_{i,K-1}), \quad (15)$$

554 并把编码、解码、logit 预测、监督信号以及后续 refinement 全部限制在这些显式槽位上。对
555 应的候选边集合写为

$$\mathcal{E}_{\text{cand}} = \{(i, j_{i,r}) \mid i \in \mathcal{V}, r \in \{0, \dots, K-1\}\}. \quad (16)$$

556 两类任务共享同一个节点级输入抽象：

$$\mathbf{s}_i = \begin{cases} [\hat{x}_i, \hat{y}_i], & \text{TSP,} \\ [\hat{x}_i, \hat{y}_i, \rho_i], & \text{CVRP,} \end{cases} \quad \rho_i = \begin{cases} d_i/C, & i \neq 0, \\ -1, & i = 0. \end{cases} \quad (17)$$

557 其中 $(\hat{x}_i, \hat{y}_i)$ 是归一化坐标。实现上，我们先对整张图做中心化，再用节点坐标平均 ℓ_2 范数
558 做缩放。TSP 的解是单条 Hamiltonian tour；CVRP 的解是以 depot 为边界的一组 routes。
559 二者的共同点在于：模型从不直接输出完整路径，而是输出同一个 row-slot 域上的 sparse
560 structural prior。

561 B.2 模型输入、噪声构造与 shuffle

562 训练时，模型接收节点特征 $\{\mathbf{s}_i\}$ 、候选槽位表 $\{\mathcal{I}_i\}$ 、时间步 t ，以及一个位于同一 row-slot
563 域中的当前结构热图 $\mathcal{H}_t$ 。TSP 与 CVRP 的主要区别，体现在 $\mathcal{H}_t$ 的构造方式上。

564 **TSP** TSP 中所有节点都是普通城市节点，不需要额外 depot 槽位。我们把目标 tour 投影
565 为 row-wise sparse adjacency $Y(\cdot)$ ，然后从目标访问顺序出发，通过随机排列或若干次随机
566 2-opt shuffle 构造噪声顺序 σ^{noise} ，并可选地再做一次整体反转，以减弱方向偏置。若训练
567 中启用 target disruption，则对 teacher 顺序再做一次额外 shuffle，得到 $\tilde{\sigma}^*$ 。于是，输入给
568 解码器的 TSP 当前状态写为

$$\mathcal{H}_t^{\text{TSP}} = (1-t)Y(\sigma^{\text{noise}}) + tY(\tilde{\sigma}^*). \quad (18)$$

569 这里所有对象都直接表示在 $n \times K$ 槽位张量上，而不会为了表达中间结构再去构造 $n \times n$
570 adjacency。

571 **CVRP** 对原生 CVRP 图，设 depot 编号为 0，客户数为 n ，因此完整实例共有 $n+1$ 个
572 节点。与 TSP 不同，我们对每个客户行显式保留一个 depot 槽位：

$$\mathcal{I}_i^{\text{CVRP}} = (0, j_{i,1}, j_{i,2}, \dots, j_{i,K-1}), \quad i \in \{1, \dots, n\}. \quad (19)$$

573 也就是说，客户行的第 0 个槽位始终对应 depot，其余槽位对应局部客户候选。CVRP 的
574 当前状态由两部分组成。第一部分是客户子图噪声：它与 TSP 一样，来自随机排列、随机
575 2-opt shuffle、可选的整体反转，以及可选的 target disruption。第二部分是 depot bias：我
576 们在距离 depot 最近的一小簇客户中采样若干个节点，构造一个偏向 depot 连接的稀疏边
577 模式 D^{bias} 。于是，

$$\mathcal{H}_{\text{cust}}^{\text{CVRP}} = (1-t)Y(\sigma^{\text{noise}}) + tY(\tilde{\sigma}^*), \quad (20)$$

578

$$\mathcal{H}_{\text{dep}}^{\text{CVRP}} = (1-t)D^{\text{bias}} + tD^*, \quad (21)$$

579

$$\mathcal{H}_t^{\text{CVRP}} = \mathcal{H}_{\text{cust}}^{\text{CVRP}} + \mathcal{H}_{\text{dep}}^{\text{CVRP}}, \quad (22)$$

580 其中 D^* 表示目标解中的 customer-to-depot 边模式。这样，CVRP 的 route-boundary 信
581 息从输入阶段就被压入统一的稀疏槽位表示。

582 B.3 整体结构：编码器、解码器与共享骨干

583 整体结构可以概括为“坐标/需求特征编码器 + 时间条件化稀疏解码器 + row-wise sparse
584 logits 头”。给定节点输入，模型先用线性层把原始特征投影到隐藏空间：

$$\mathbf{h}_i^{(0)} = W_{\text{in}} \mathbf{s}_i. \quad (23)$$

585 随后，编码器只在 $\mathcal{I}_i$ 给出的局部候选上执行多层 sparse multi-head attention, 从而在 $O(nK)$
586 而非 $O(n^2)$ 的复杂度下传播全局信息，其中 n 表示当前 row-slot 表示中的行数：

$$\mathbf{h}_i^{(\ell+1)} = \text{EncLayer}^{(\ell)}\left(\mathbf{h}_i^{(\ell)}, \{\mathbf{h}_{j_{i,r}}^{(\ell)}\}_{r=0}^{K-1}\right). \quad (24)$$

587 编码器输出经过一个中间投影后，与时间步嵌入相加，再送入解码器：

$$\mathbf{z}_i^{(0)} = W_m \mathbf{h}_i + \mathbf{e}(t). \quad (25)$$

588 解码器与编码器共享同一套 row-slot 邻接抽象，但额外把当前结构热图 $\mathcal{H}_t$ 作为 attention
589 bias 注入：

$$\mathbf{z}_i^{(\ell+1)} = \text{DecLayer}^{(\ell)}\left(\mathbf{z}_i^{(\ell)}, \{\mathbf{z}_{j_{i,r}}^{(\ell)}\}_{r=0}^{K-1}; \mathcal{H}_t\right). \quad (26)$$

590 最后，一个轻量投影头产生用于构造 sparse logits 的最终特征

$$\mathbf{g}_i = \psi(\mathbf{z}_i). \quad (27)$$

591 因此，编码器负责在稀疏候选图上汇聚全局上下文，解码器负责把时间步与当前结构状态注
592 入到同一个 row-slot 域中，整个主干始终不离开 sparse-only 的表示。

593 B.4 模型输出与损失函数

594 模型输出不是完整路径，而是每一行候选槽位上的 sparse logits。对节点 i 的第 r 个槽位，
595 我们直接用终层特征与其候选节点特征的点积构造打分：

$$\ell_{i,r} = \langle \mathbf{g}_i, \mathbf{g}_{j_{i,r}} \rangle, \quad p_{i,r}^{(\theta)} = \frac{\exp(\ell_{i,r})}{\sum_{u=0}^{K-1} \exp(\ell_{i,u})}. \quad (28)$$

596 实现中，我们显式屏蔽自环槽位，并把最终概率直接交给下游 candidate refinement；因此，
597 整个输出接口仍然是 row-wise sparse 的，而不是稠密路径解码器。

598 **TSP** 对 TSP，我们把并行搜索得到的多条高质量 tour $\{\tau^{(m)}\}_{m=1}^M$ 一起投影到同一个候选
599 表上。对节点 i ，只要某个槽位在任意一条 tour 中作为其前驱或后继出现，就进入支持集

$$\mathcal{U}_i^{\text{TSP}} = \{r \mid \exists m, j_{i,r} \in \{\text{prev}_{\tau^{(m)}}(i), \text{next}_{\tau^{(m)}}(i)\}\}. \quad (29)$$

600 于是，TSP 的监督目标是逐行 soft label

$$y_{i,r}^{\text{TSP}} = \begin{cases} \frac{1}{|\mathcal{U}_i^{\text{TSP}}|}, & r \in \mathcal{U}_i^{\text{TSP}}, \\ 0, & \text{otherwise,} \end{cases} \quad \sum_{r=0}^{K-1} y_{i,r}^{\text{TSP}} = 1. \quad (30)$$

601 对应损失就是 row-wise sparse cross-entropy,

$$\mathcal{L}_{\text{TSP}} = -\frac{1}{n} \sum_{i=1}^n \sum_{r=0}^{K-1} y_{i,r}^{\text{TSP}} \log p_{i,r}^{(\theta)}. \quad (31)$$

602 这意味着 TSP 学到的不是单条 tour 的二值标签，而是多解投影得到的 sparse soft target。

603 **CVRP** 对 CVRP，监督同样来自并行搜索得到的多条高质量 route sets $\{\mathcal{R}^{(m)}\}_{m=1}^M$ ，但
604 支持集还要显式纳入 depot 边界信号：

$$\mathcal{U}_i^{\text{CVRP}} = \left\{ r \mid \begin{array}{l} \exists m, j_{i,r} = \text{prev}_{\mathcal{R}^{(m)}}(i) \\ \text{或 } j_{i,r} = \text{next}_{\mathcal{R}^{(m)}}(i) \\ \text{或 } (j_{i,r} = 0 \wedge i \text{ 为 route-boundary customer}) \end{array} \right\}. \quad (32)$$

因此, CVRP 的 soft target 写为

$$y_{i,r}^{\text{CVRP}} = \begin{cases} \frac{1}{|\mathcal{U}_i^{\text{CVRP}}|}, & r \in \mathcal{U}_i^{\text{CVRP}}, \\ 0, & \text{otherwise,} \end{cases} \quad \sum_{r=0}^{K-1} y_{i,r}^{\text{CVRP}} = 1. \quad (33)$$

为了强调 customer-to-depot 正样本, 我们使用带权 sparse loss:

$$\omega_{i,r}^{\text{CVRP}} = \begin{cases} \lambda_{\text{dep}}, & y_{i,r}^{\text{CVRP}} > 0 \text{ 且 } j_{i,r} = 0, \\ 1, & \text{otherwise,} \end{cases} \quad (34)$$

$$\mathcal{L}_{\text{CVRP}} = -\frac{1}{n} \sum_{i=1}^n \sum_{r=0}^{K-1} \omega_{i,r}^{\text{CVRP}} y_{i,r}^{\text{CVRP}} \log p_{i,r}^{(\theta)}. \quad (35)$$

此外, depot 槽位的概率还会在推理阶段被继续消费, 用来估计 boundary customers 并在 sparse candidate graph 中决定需要实例化多少 depot copies; 整个过程中都不会显式回退到稠密的 $(N+E) \times (N+E)$ 图。

B.5 稀疏多头注意力 Triton kernel 与 online softmax

在实现上, 稀疏编码器和解码器都调用同一个 sparse multi-head attention 接口。对 head h 、节点行 i 以及槽位 r , 我们有

$$\mathbf{q}_i^{(h)} = W_Q^{(h)} \mathbf{z}_i, \quad \mathbf{k}_{i,r}^{(h)} = W_K^{(h)} \mathbf{z}_{j_{i,r}}, \quad \mathbf{v}_{i,r}^{(h)} = W_V^{(h)} \mathbf{z}_{j_{i,r}}, \quad (36)$$

$$e_{i,r}^{(h)} = \frac{\langle \mathbf{q}_i^{(h)}, \mathbf{k}_{i,r}^{(h)} \rangle}{\sqrt{d_h}} + b_{i,r}^{(h)}. \quad (37)$$

这里的 $b_{i,r}^{(h)}$ 统一吸收无效槽位 mask、当前状态 bias, 以及 TSP/CVRP 特有的结构偏置。Triton kernel 不会 materialize 一整行 softmax, 而是按槽位块维护 running maximum 与 running partition sum:

$$m_i^{(h,t)} = \max \left(m_i^{(h,t-1)}, \max_{r \in \mathcal{B}_t} \hat{e}_{i,r}^{(h,t)} \right), \quad m_i^{(h,0)} = -\infty, \quad (38)$$

$$\lambda_i^{(h,t)} = \lambda_i^{(h,t-1)} \exp(m_i^{(h,t-1)} - m_i^{(h,t)}) + \sum_{r \in \mathcal{B}_t} \exp(\hat{e}_{i,r}^{(h,t)} - m_i^{(h,t)}), \quad \lambda_i^{(h,0)} = 0. \quad (39)$$

其中 $\hat{e}_{i,r}^{(h,t)}$ 是块内 masked score。基于同样的 online softmax 统计量, kernel 在一次遍历中同步维护 value aggregation:

$$\mathbf{o}_i^{(h,t)} = \frac{\lambda_i^{(h,t-1)} \exp(m_i^{(h,t-1)} - m_i^{(h,t)})}{\lambda_i^{(h,t)}} \mathbf{o}_i^{(h,t-1)} + \frac{1}{\lambda_i^{(h,t)}} \sum_{r \in \mathcal{B}_t} \exp(\hat{e}_{i,r}^{(h,t)} - m_i^{(h,t)}) \mathbf{v}_{i,r}^{(h)}. \quad (40)$$

最终得到

$$\alpha_{i,r}^{(h)} = \frac{\exp(e_{i,r}^{(h)} - m_i^{(h,T)})}{\lambda_i^{(h,T)}}. \quad (41)$$

与 JAX reference 实现相比, Triton 版本进一步把多个相邻 query 行和多个 attention heads 融合到同一个 kernel 中, 共享 ‘neighbor_idx’ 和 bias 的读取; 同时, 我们还对 sparse logits 和 sparse cross-entropy 使用了独立的 Triton kernel, 使 gather-dot、row-wise normalization 与 loss 计算始终停留在同一个 $n \times K$ 槽位域中。这也是本文能够在 sparse-only 主干上高效运行的关键。

C 基线方法的实现细节

本节仅说明表 ??、表 ?? 和表 ?? 中实际出现的基线方法。具体包括：(1) **经典求解器**：Concorde [?]、LKH3 [?] 和 HGS [?]; (2) **构造式 NCO 方法**：POMO [?]、BQ [?]、LEHD [?]、INViT [?]、ELG [?], 以及 L2C-Insert 的 Greedy 和 $I=1000$ 两个变体; (3) **热图式 NCO 方法**：Att-GCN+MCTS [?]、DIFUSCO [?]、T2T [?]、Fast-T2T 和 GenSCO; (4) **两阶段方法**：H-TSP [?]、GLOP [?]、GLOP-G (LKH3) 以及 UDC, 其中表 ?? 和表 ?? 使用 UDC- $\mathbf{x}_{50}$, 表 ?? 使用 UDC- $\mathbf{x}_{250}$ 。未在这三张主表中出现的方法不在本节展开。

对于表 ?? 和表 ?? 中带星号 (*) 的结果, 我们直接采用原论文报告值。除特别说明外, 其余基线均使用公开实现, 并遵循各自论文或代码仓库的默认评测设置; 表格中的方法名称 (如 “POMO aug $\times 8$ ”、“BQ bs16”、“LEHD RRC1000”、“INViT greedy”、“L2C-Insert Greedy” 和 “L2C-Insert ($I=1000$)”) 直接对应所使用的推理配置。

- **Concorde** [?] 运行 Concorde 的 Python 封装 PyConcorde, 在各规模 TSP 实例上使用默认设置。
- **LKH3** [?] 运行 AM [?] 提供的代码。对于 TSP100K, 遵循 [?] 的建议, 使用 POPMUSIC [?] 降低预处理复杂度 (设置 CANDIDATE SET TYPE = POPMUSIC 和 INITIAL PERIOD=1000), 其他设置默认。
- **HGS** [?] 运行 HGS-CVRP 求解器的 Python 封装 PyHygese。
- **LEHD** [?] 在观察到将 RRC 的最大破坏规模限制为 1000 时性能几乎不变 (而不限制时推理时间显著增加) 后, 将其最大破坏规模限制为 1000。
- **L2C-Insert** 在主表中同时报告贪心解码版本和带局部重构的 $I=1000$ 版本, 因此分别记为 “L2C-Insert Greedy” 与 “L2C-Insert ($I=1000$)”。
- **UDC** 在合成数据主表中使用 UDC- $\mathbf{x}_{50}$ ($\alpha=50$), 在真实世界主表中使用 UDC- $\mathbf{x}_{250}$ ($\alpha=50$); 我们在表中保留原论文对应的模型记号。

D 补充实验

本节汇报补充材料。我们首先给出真实世界 TSPLIB、VLSI 与 Art-TSP 上的补充 TSP 结果, 并补充 CVRPLIB 上的真实世界 CVRP 结果。对于 TSPLIB, 我们整理 GenSCO [?] 中的 baseline, 并补充 LSL 在 Guided-LKH t512 设定下的 gap 与 Time(s) 列; 对于 VLSI, 我们按照小于 2000 与 2000–200K 两个规模区间整理结果表; 对于 Art-TSP, 我们进一步汇报超大规模实例上的 Guided-LKH 结果; 对于 CVRPLIB, 我们按小于 2000 与 2000–20K 两个规模区间列出过滤掉 ORTEC、Loggi 与 Belgium 三个扩展集后的实例名称。其余附录部分则进一步讨论方法定位、复杂度分析以及代表性可视化示例。

D.1 TSPLIB 真实世界 TSP 结果

表 ?? 和表 ?? 汇报 TSPLIB 真实世界 TSP 的补充结果。其中 AM*、GCN*、DIFUSCO、Fast T2T 和 GenSCO 等 baseline 列根据 GenSCO [?] 的 Table 7 和 Table 8 整理; 新增的 L2C-Insert 列来自本地评测结果, NeuroLKH 列则根据 NeuroLKH [?] 的 TSPLIB 补充表整理; 最后两列给出我们在 Guided-LKH t512 设定下的 LSL gap 与 Time(s)。在 50–200 节点的 29 个实例上, LSL 的平均 gap 为 0.000%, 平均 Time(s) 为 11.77。在更大的 200–1000 节点组中, 表中共包含 19 个实例; 此时 LSL 的平均 gap 为 0.008%, 平均 Time(s) 为 23.99, 仅在 f1417、u574 和 p654 上出现可见但仍很小的非零 gap (分别为 0.035%、0.093% 和 0.025%)。整体上, 真实世界 TSPLIB 的 Time(s) 随规模增大而上升, 但在 t512 设定下仍保持了接近最优的解质量。

D.2 VLSI 真实世界 TSP 结果

表 ?? 和表 ?? 汇报 VLSI 真实世界 TSP 的补充结果, 均根据对应的真实世界结果汇总文件整理。前者覆盖小于 2000 节点的实例, 并报告 GenSCO、L2C-Insert、NeuroLKH、LSL Gap 与 LSL Time(s)。后者覆盖 2000–200K 节点的实例, 并报告 L2C-Insert、NeuroLKH、LSL Gap 与 LSL Time(s)。

Table 5: TSPLIB 中 50–200 节点真实世界 TSP 实例上的结果。AM*、GCN*、DIFUSCO、Fast T2T 和 GenSCO 列根据 GenSCO [?] 的 Table 7 整理；‘*’ 表示该结果在源论文中引用自更早工作。最后两列给出本文在 Guided-LKH t512 设定下的 LSL 结果。

Instances	AM*	GCN*	DIFUSCO	Fast T2T	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
eil51	16.767%	40.025%	2.82%	0.00%	0.000%	0.000%	0.000%	0.000%	7.44
berlin52	4.169%	33.225%	0.00%	0.00%	0.000%	1.584%	0.000%	0.000%	7.82
st70	1.737%	24.785%	0.00%	0.00%	0.000%	0.000%	0.000%	0.000%	8.73
eil76	1.992%	27.411%	0.34%	0.00%	0.000%	0.004%	0.000%	0.000%	8.69
pr76	0.816%	27.793%	1.12%	0.00%	0.000%	0.000%	0.000%	0.000%	8.25
rat99	2.645%	17.633%	0.09%	0.00%	0.000%	12.416%	0.000%	0.000%	8.63
kroA100	4.017%	28.828%	0.10%	0.00%	0.000%	5.013%	0.000%	0.000%	8.60
kroB100	5.142%	34.686%	2.29%	0.65%	0.000%	5.300%	0.000%	0.000%	8.62
kroC100	0.972%	35.506%	0.00%	0.00%	0.000%	6.076%	0.000%	0.000%	8.59
kroD100	2.717%	38.018%	0.07%	0.00%	0.000%	4.815%	0.000%	0.000%	8.79
kroE100	1.470%	26.589%	3.83%	0.00%	0.000%	4.050%	0.000%	0.000%	8.75
rd100	3.407%	50.432%	0.08%	0.00%	0.000%	0.000%	0.000%	0.000%	8.53
eil101	2.994%	26.701%	0.03%	0.00%	0.000%	0.000%	0.000%	0.000%	8.30
lin105	1.739%	34.902%	0.00%	0.00%	0.000%	4.337%	0.000%	0.000%	7.88
pr107	3.933%	80.564%	0.91%	0.62%	0.000%	0.000%	0.000%	0.000%	9.31
pr124	3.677%	70.146%	1.02%	0.08%	0.000%	0.000%	0.000%	0.000%	10.06
bier127	5.908%	45.561%	0.94%	1.50%	0.000%	0.000%	0.016%	0.000%	10.17
ch130	3.182%	39.090%	0.29%	0.00%	0.000%	0.000%	0.000%	0.000%	11.64
pr136	5.064%	58.673%	0.19%	0.01%	0.000%	0.000%	0.000%	0.000%	13.86
pr144	7.641%	55.837%	0.80%	0.39%	0.000%	0.000%	0.081%	0.000%	15.66
ch150	4.584%	49.743%	0.57%	0.00%	0.000%	0.000%	0.000%	0.000%	11.69
kroA150	3.784%	45.411%	0.34%	0.00%	0.000%	3.220%	0.000%	0.000%	11.86
kroB150	2.437%	56.745%	0.30%	0.07%	0.340%	3.846%	0.000%	0.000%	12.75
pr152	7.494%	33.925%	1.69%	0.19%	0.187%	0.005%	0.037%	0.000%	30.35
u159	7.551%	38.338%	0.82%	0.00%	0.000%	0.025%	0.000%	0.000%	11.40
rat195	6.893%	24.968%	1.48%	0.79%	0.194%	4.828%	0.000%	0.000%	14.72
d198	373.020%	62.351%	3.32%	0.86%	0.751%	1.835%	0.285%	0.000%	35.11
kroA200	7.106%	40.885%	2.28%	0.49%	0.160%	1.025%	0.000%	0.000%	12.67
kroB200	8.541%	43.643%	2.35%	2.50%	0.098%	0.219%	0.000%	0.000%	12.39
Mean	16.767%	40.025%	0.97%	0.28%	0.061%	2.021%	0.014%	0.000%	11.77

Table 6: TSPLIB 中 200–1000 节点真实世界 TSP 实例上的结果。DIFUSCO、Fast T2T 和 GenSCO 列根据 GenSCO [?] 的 Table 8 整理。最后两列给出本文在 Guided-LKH t512 设定下的 LSL 结果。

Instances	DIFUSCO	Fast T2T	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
a280	1.39%	0.10%	0.000%	0.109%	0.000%	0.000%	18.76
d493	1.81%	1.43%	3.872%	0.469%	0.086%	0.000%	21.35
d657	4.86%	0.64%	0.457%	0.751%	0.001%	0.000%	20.16
fl417	3.30%	2.01%	0.596%	0.408%	0.056%	0.035%	46.46
gil262	2.18%	0.18%	0.000%	0.089%	0.000%	0.000%	13.53
lin318	2.95%	1.21%	0.025%	0.038%	0.000%	0.000%	13.96
p654	7.49%	1.67%	1.023%	1.815%	0.354%	0.025%	96.24
pcb442	2.59%	0.61%	0.015%	0.598%	0.000%	0.000%	15.95
pr226	4.22%	0.34%	0.211%	0.290%	0.016%	0.000%	14.11
pr264	0.92%	0.73%	0.000%	0.000%	0.000%	0.000%	35.31
pr299	1.46%	1.40%	0.011%	2.390%	0.000%	0.000%	13.90
pr439	2.73%	0.50%	0.106%	0.387%	0.047%	0.000%	22.11
rat575	2.32%	1.43%	0.000%	0.825%	0.001%	0.000%	18.61
rat783	3.04%	1.03%	0.068%	0.995%	0.000%	0.000%	20.82
rd400	1.18%	0.08%	0.030%	0.162%	0.000%	0.000%	14.52
ts225	4.95%	1.37%	0.517%	0.000%	0.000%	0.000%	12.48
tsp225	3.25%	0.81%	0.000%	2.860%	0.000%	0.000%	12.10
u574	2.50%	0.94%	0.109%	0.630%	0.000%	0.093%	17.05
u724	2.05%	1.41%	0.040%	0.555%	0.000%	0.000%	28.36
Mean	2.905%	0.942%	0.373%	0.704%	0.030%	0.008%	23.99

674 D.3 Art-TSP 真实世界 TSP 结果

675 表 ?? 汇报 6 个 Art-TSP 超大规模真实世界 TSP 实例上的结果。在这 6 个实例上, LSL 的
676 gap 均保持在 0.025%–0.032% 之间, wall-clock Time(s) 则随着规模从 100K 增长到 200K
677 而由 12105.279 增加到 24469.262。

Table 7: VLSI 中 小于 2000 节点真实世界 TSP 实例上的结果。

Instance	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)	Instance	GenSCO	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
xqf131	-0.1378%	4.7647%	0.00%			pbd984	-0.2248%	0.3784%	0.00%		
xqg237	0.0431%	1.2889%	0.00%			xit1083	-0.2262%	0.6737%	0.00%		
pma343	0.6107%	15.2290%	0.00%			dka1376	-0.1229%	1.6946%	0.00%		
pka379	1.7708%	19.8753%	0.00%			dca1389	0.4114%	0.5395%	0.00%		
bcl380	-0.1926%	-0.1342%	0.00%			dja1436	0.4369%	1.0077%	0.00%		
pbl395	-0.3638%	-0.3565%	0.00%			icw1483	0.2263%	1.2454%	0.00%		
pbl411	-0.4599%	-0.3756%	0.00%			fra1488	3.3967%	0.6374%	0.00%		
pbn423	-0.3832%	-0.3395%	0.00%			rbv1583	0.5977%	1.1684%	0.00%		
pbn436	-0.3636%	0.1982%	0.00%			rby1599	0.6602%	0.9665%	0.00%		
xql662	-0.1125%	0.0279%	0.00%			fnb1615	1031.5743%	1.0392%	0.02%		
rbx711	-0.1289%	0.9480%	0.00%			djc1785	2.2686%	0.9319%	0.00%		
rbu737	0.2230%	0.7630%	0.00%			dcc1911	0.7610%	1.1018%	0.02%		
dkg813	0.8194%	0.6660%	0.00%			dkd1973	23.2987%	0.6533%	0.05%		
lim963	-0.0551%	0.7854%	0.00%								

Table 8: VLSI 中 2000–200K 节点真实世界 TSP 实例上的结果。

Instance	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
djb2036	0.4914%	0.00%	-0.331%	359.691	ido21215	3.4863%		-0.261%	1359.514
dcb2086	1.0685%	0.00%	-0.192%	338.146	fma21553	3.7856%		-0.237%	1247.462
bva2144	0.8540%	0.00%	-0.221%	433.101	lsb22777	4.2038%		-0.291%	1376.764
xqc2175	1.5891%	0.00%	-0.214%	496.137	xrh24104	5.3299%		-0.242%	2226.190
bck2217	0.7445%	0.01%	-0.330%	1387.759	bbz25234	4.9082%		-0.318%	2102.744
xpr2308	1.2130%	0.00%	-0.239%	1147.541	irx28268	4.2582%		-0.363%	2092.812
ley2323	1.1626%	0.10%	-0.233%	1147.441	fyg28534	7.7193%		-0.282%	1937.243
irw2802	1.5471%	0.00%	-0.253%	1147.305	icx28698	6.6384%		-0.294%	1937.086
dbj2924	1.3585%	0.00%	-0.137%	1147.130	boa28924	6.4090%		-0.295%	1612.595
pia3056	2.2171%	0.00%	-0.339%	942.189	ird29514	5.4607%		-0.225%	1803.827
xqc3891	1.2462%	0.02%	-0.183%	1040.456	pbb30440	4.9287%		-0.186%	1244.193
bgb4355	1.4532%	0.06%	-0.333%	930.654	xib32892	5.4544%		-0.257%	1555.981
xsc6880	2.0405%	0.02%	-0.374%	1246.073	fry33203	7.1948%		-0.187%	4260.619
lap7454	3.0127%	0.05%	-0.328%	1226.154	bby34656	6.3148%		-0.291%	4260.516
ida8197	2.1110%	0.16%	-0.300%	2084.140	pba38478	6.5314%		-0.260%	3958.797
xmc10150	2.4490%	0.10%	-0.295%	2835.700	ics39603	9.4781%		-0.315%	4115.049
xvb13584	2.3437%		-0.276%	2956.059	rbz43748	7.4829%		-0.309%	4114.862
xrb14233	3.2484%		-0.120%	2380.095	fhf47608	8.6438%		-0.261%	3958.690
xia16928	3.7517%		0.197%	3310.673	fna52057	7.3344%		-0.276%	4260.355
pjh17845	3.0073%		0.023%	3077.562	bna56769	6.7615%		-0.331%	3958.543
frh19289	3.9645%		-0.102%	2955.976	dan59296	9.1025%		-0.249%	3783.674
fnc19402	3.8160%		0.020%	2276.261	sra104815	6.1138%		-0.701%	9706.833

Table 9: Art-TSP 超大规模真实世界 TSP 实例上的结果。

Instance	L2C-Insert	NeuroLKH	LSL Gap	LSL Time(s)
mona-lisa100K	3.7640%		0.029%	12105.279
vangogh120K	4.0220%		0.031%	14898.313
venus140K	4.1016%		0.025%	17860.376
pareja160K	4.2593%		0.027%	19513.381
courbet180K	4.7775%		0.027%	22618.978
earring200K	4.5136%		0.032%	24469.262

678 D.4 CVRPLIB 真实世界 CVRP 结果

679 表 ?? 和表 ?? 汇报 CVRPLIB 真实世界 CVRP 的补充结果。实例名称直接取自本
680 地 `test_dataset/cvrp/cvrplib/instances` 目录下对应的 `.vrp` 文件名, 并先过滤掉
681 ORTEC、Loggi 与 Belgium 三个扩展集。现在, 表 ?? 仅保留 100 个 `set-X` 实例 (从
682 `X-n101-k25` 到 `X-n1001-k43`); 表 ?? 则汇总全部 100 个 `set-XL` 实例 (从 `XL-n1048-k237`
683 到 `XL-n10001-k1570`), 其中也包括原先混排在小规模表中的小于 2000 节点 `set-XL` 实例。
684 按照当前补充实验的展示口径, 我们在每个实例后统一保留 L2C-Insert、NeuroLKH、LSL
685 Gap 与 LSL Time(s) 四列。为压缩纵向篇幅, 这两张表均采用正常竖版页面中的三栏并排
686 列布局, 字体大小与 TSPLIB 50-200 节点结果表保持一致。

Table 10: CVRPLIB 中 **set-X** 真实世界 CVRP 实例上的结果。

Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)
X-n101-k25		0.722%			X-n261-k13		2.825%			X-n491-k59				
X-n106-k14		0.632%			X-n266-k58		1.464%			X-n502-k39				
X-n110-k13		0.689%			X-n270-k35		1.492%			X-n513-k21				
X-n115-k10		0.289%			X-n275-k28		1.257%			X-n524-k153				
X-n120-k6		0.433%			X-n280-k17		2.016%			X-n536-k96				
X-n125-k30		0.797%			X-n284-k15		3.084%			X-n548-k50				
X-n129-k18		0.982%			X-n289-k60		1.942%			X-n561-k42				
X-n134-k13		1.178%			X-n294-k50		1.598%			X-n573-k30				
X-n139-k10		0.829%			X-n298-k31		2.578%			X-n586-k159				
X-n143-k7		1.184%			X-n303-k21					X-n599-k92				
X-n148-k46		0.852%			X-n308-k13					X-n613-k62				
X-n153-k22		0.429%			X-n313-k71					X-n627-k43				
X-n157-k13		0.171%			X-n317-k53					X-n641-k35				
X-n162-k11		0.759%			X-n322-k28					X-n655-k131				
X-n167-k10		1.463%			X-n327-k20					X-n670-k130				
X-n172-k51		0.850%			X-n331-k15					X-n685-k75				
X-n176-k26		0.978%			X-n336-k84					X-n701-k44				
X-n181-k23		0.457%			X-n344-k43					X-n716-k35				
X-n186-k15		1.566%			X-n351-k40					X-n733-k159				
X-n190-k8		1.648%			X-n359-k29					X-n749-k98				
X-n195-k51		1.047%			X-n367-k17					X-n766-k71				
X-n200-k36		0.899%			X-n376-k94					X-n783-k48				
X-n204-k19		1.496%			X-n384-k52					X-n801-k40				
X-n209-k16		1.579%			X-n393-k38					X-n819-k171				
X-n214-k11		5.916%			X-n401-k29					X-n837-k142				
X-n219-k73		0.118%			X-n411-k19					X-n856-k95				
X-n223-k34		1.449%			X-n420-k130					X-n876-k59				
X-n228-k23		1.121%			X-n429-k61					X-n895-k37				
X-n233-k16		1.911%			X-n439-k37					X-n916-k207				
X-n237-k14		1.334%			X-n449-k29					X-n936-k151				
X-n242-k48		1.200%			X-n459-k26					X-n957-k87				
X-n247-k50		0.979%			X-n469-k138					X-n979-k58				
X-n251-k28		1.488%			X-n480-k70					X-n1001-k43				
X-n256-k16		3.158%												

Table 11: CVRPLIB 中 **set-XL** 真实世界 CVRP 实例上的结果。

Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)	Instance	L2C-Insert	Neuro LKH	LSL Gap	LSL Time(s)
XL-n1048-k237					XL-n2634-k17					XL-n4951-k203				
XL-n1094-k157					XL-n2681-k540					XL-n5061-k184				
XL-n1141-k112					XL-n2727-k546					XL-n5174-k55				
XL-n1188-k96					XL-n2774-k286					XL-n5288-k1246				
XL-n1234-k55					XL-n2821-k208					XL-n5406-k783				
XL-n1281-k29					XL-n2867-k120					XL-n5526-k553				
XL-n1328-k19					XL-n2914-k95					XL-n5649-k401				
XL-n1374-k278					XL-n2961-k55					XL-n5774-k290				
XL-n1421-k232					XL-n3007-k658					XL-n5902-k122				
XL-n1468-k151					XL-n3054-k461					XL-n6034-k61				
XL-n1514-k106					XL-n3101-k311					XL-n6168-k1922				
XL-n1561-k75					XL-n3147-k232					XL-n6305-k1042				
XL-n1608-k39					XL-n3194-k161					XL-n6445-k628				
XL-n1654-k11					XL-n3241-k115					XL-n6588-k473				
XL-n1701-k562					XL-n3287-k30					XL-n6734-k330				
XL-n1748-k271					XL-n3334-k934					XL-n6884-k148				
XL-n1794-k163					XL-n3408-k524					XL-n7037-k38				
XL-n1841-k126					XL-n3484-k436					XL-n7193-k1683				
XL-n1888-k82					XL-n3561-k229					XL-n7353-k1471				
XL-n1934-k46					XL-n3640-k211					XL-n7516-k859				
XL-n1981-k13					XL-n3721-k77					XL-n7683-k602				
XL-n2028-k617					XL-n3804-k29					XL-n7854-k365				
XL-n2074-k264					XL-n3888-k1010					XL-n8028-k294				
XL-n2121-k186					XL-n3975-k687					XL-n8207-k108				
XL-n2168-k138					XL-n4063-k347					XL-n8389-k2028				
XL-n2214-k131					XL-n4153-k291					XL-n8575-k1297				
XL-n2261-k54					XL-n4245-k203					XL-n8766-k1032				
XL-n2307-k34					XL-n4340-k148					XL-n8960-k634				
XL-n2354-k631					XL-n4436-k48					XL-n9160-k379				
XL-n2401-k408					XL-n4535-k1134					XL-n9363-k209				
XL-n2447-k290					XL-n4635-k790					XL-n9571-k55				
XL-n2494-k194					XL-n4738-k487					XL-n9784-k2774				
XL-n2541-k121					XL-n4844-k321					XL-n10001-k1570				
XL-n2587-k66														

687 E 为什么 LSL 将全局预测与离散搜索分开

688 LSL 的最终性能来自两个组成部分的协同：一个保留全局信息的稀疏预测骨干，以及一个
689 离散的启发式精炼后端。核心设计选择在于，我们将这两类职责分开，而不是把它们全部塞
690 进一个重型 constructive decoder 中。

691 **为什么不把所有事情都交给 decoder** 旧的构造式范式通常要求神经模型在解构造过程中
692 同时承担三件事：判断全局结构、满足组合约束、以及做出细粒度离散落点决策。与此同时，
693 许多已有方法在训练时仍主要依赖单条参考解或近似一热的尖锐热图监督。在小规模问题
694 上，这种目标有时仍然有效，因为较高质量甚至最优解相对容易获得，模型可以借助这些监
695 督隐式学到更偏向低 cost 结构的倾向。

696 但在大规模问题上，情况会发生变化。首先，高质量监督显著更难获得，训练数据中更容易
697 混入质量不稳定、却又不容易被显式识别的次优解或离群解。若仍采用单解、尖锐的监督目
698 标，这些彼此冲突的信号就会被直接压到模型上，既会削弱训练稳定性，也会让模型过早地
699 把概率质量锁定在某一条具体路径上。其次，当部分解越来越复杂时，decoder 还必须同时
700 处理尚未解决的未节点、长距离几何耦合以及可行性约束，这会进一步放大神经构造式范
701 式在计算和学习上的负担。

702 **LSL 为什么不同** LSL 通过让稀疏骨干负责全局感知的候选评分、让启发式后端负责离散
703 可行性与迭代改进，避免了这一瓶颈。模型不再被要求一次性吐出最终可行解，也不必把
704 “学习低 cost 偏好”和“隐式学习完整约束满足”这两类任务全部压缩到单一 decoder 内部。
705 相反，神经模块只需输出对后续搜索真正有用的结构先验，而离散后端负责把这些偏好转化
706 为可行且不断改进的解。

707 这类分解还有一个更重要的作用：它避免了单解监督把搜索空间过早锁死。LSL 在训练时
708 并不把所有节点都压成唯一“正确边”，而是让多条高质量搜索解共同形成逐行软目标。于
709 是，在结构稳定的位置，模型可以学到明确偏好；而在本身存在多个合理局部替代的位置，
710 模型会保留必要的不确定性，为后续搜索留下可探索空间。从搜索角度看，这也意味着系统
711 不必把每一步中间状态都严格等同于最终监督中的那条单一可行解，而可以通过更灵活的
712 离散精炼过程逐步逼近更优解。正是这种分解，使框架在大规模问题上更可扩展：模型不需
713 要在自回归式构造过程中独自解决整个组合优化问题，而搜索后端仍能获得一个足够强、且
714 对搜索更友好的学习先验来聚焦后续精炼。

715 F 行级软目标支持集与交叉熵损失的合理性

716 下面进一步说明 LSL 中监督目标的定义方式，以及基于公式 ?? 的行级交叉熵损失为何是
717 自然的选择。

718 **目标并不是每个节点只对应一条边** 对于每个训练实例，我们会保留并行启发式搜索得到
719 的多条高质量解，并将它们共同投影到同一稀疏候选图上。对节点 i 而言，目标支持集由所
720 有在至少一条保留高质量解中出现过的候选边组成。因此，节点 i 的监督不会被压缩成唯
721 的一条“正确边”。

722 **行级软目标是如何形成的** 记 $\mathcal{U}_i$ 为在投影后、节点 i 对应的所有被观测到的候选边集合。
723 我们对 $\mathcal{U}_i$ 中的每一条边均匀分配概率质量，对其余候选边赋值为 0，然后再做逐行归一化。
724 于是，当多条高质量解在同一局部结构上高度一致时，对应目标分布会更尖锐；而当多个局
725 部替代结构都持续出现时，对应目标分布则保持更分散。

726 **为什么这种支持集定义重要** 这种构造保留了搜索过程中揭示出来的结构不确定性。于是，
727 学习到的先验既能告诉下游求解器哪些位置应该更有把握，也能告诉它哪些位置需要为后
728 续精炼保留多个合理备选。

729 **为什么行级交叉熵损失是合理的** 公式 ?? 中的损失，本质上是预测稀疏候选分布与由多条
730 高质量搜索解诱导出的软目标之间的逐行交叉熵。这是一个自然且合理的选择，因为模型对
731 每个节点输出的本来就是候选集合上的概率分布。

732 与硬的一热目标不同，LSL 中的软目标可能会在多个候选边上分配概率质量。因此，这个逐
733 行交叉熵会同时鼓励两种行为：当搜索反复确认同一局部结构时，模型会被推动得更尖锐；
734 而当搜索持续揭示多个合理替代时，模型则会被鼓励保留必要的不确定性。

这个目标同样非常适合梯度优化。对于那些在搜索中被反复支持、但模型却赋予过低概率的候选边，损失会施加更大惩罚；而对于本身就具有歧义的区域，模型则不会被强行压成一个过度确定的一热答案。这样一来，训练目标就和模型在推理时的真实角色保持了一致：它学习的是一个适合搜索的稀疏先验，而不是一个被过早锁死的最终路径。

G 关于公平比较与成本计算的说明

关于计算预算比较的说明 我们想说明的是，直接比较“迭代次数”可能并不容易，因为不同方法中“一次迭代”的定义及其计算成本存在显著差异：

- 像 NeurOpt 这样的改进型方法在每次迭代中执行非常快速的局部搜索操作（例如 k-opt）。
- 像 GLOP 与 UDC 这样的两阶段方法在每次迭代中会调用多个子问题求解器，或在多个初始解上执行破坏与修复，并且还可能在内部调用诸如 LKH3 的强力启发式求解器。
- LSL 使用一个神经骨干和一个启发式后端，在单条稀疏先验引导的解轨迹上执行精炼。

因此，总推理时间可能更能全面且公平地反映计算预算。基于这一点，我们的分析聚焦于在给定运行时间内所达到的解质量（最优 gap）。从总运行时间的角度看，我们的实验结果支持这一观点。例如，在表 1（TSP1K）中，LSL（I=1000）在 46.4 分钟内实现了 0.481% 的 gap。相比之下，像 LEHD+RRC1000 这样的强基线需要 5.2 小时才能达到 0.731% 的 gap。这表明，我们的方法在时间与解质量之间呈现出较好的权衡。

H 总推理时间与单实例平均推理时间

较大规模实例的总推理时间之所以看起来反而下降，是因为不同问题规模的测试数据集所包含的实例数量不同。

遵循先前 NCO 研究中建立的标准评估协议 [??]，较大规模问题的测试集通常包含显著更少的实例。例如，我们的 TSP100 测试集包含 10,000 个实例，而 TSP10K 与 TSP100K 测试集则各自只包含 16 个实例。我们在主论文表 1 和表 2 中报告的总推理时间，反映的是求解整个数据集所需的时间。

为了更清晰地说明这一点，下表将总推理时间进一步拆分为单个实例的平均推理时间。

Table 12: LSL 的总推理时间与单实例平均推理时间。

数据集规模	TSP100 10000 个实例	TSP1K 128 个实例	TSP10K 16 个实例	TSP100K 16 个实例
LSL 总推理时间	8.0h	46.4m	16.64m	26.07m
LSL 单实例推理时间	2.9s	21.8s	62.4s	97.8s

如表所示，LSL 的单实例平均推理时间会随着问题规模增大而按预期上升，从 TSP100 的 2.9 秒增加到 TSP100K 的 97.8 秒。

I LSL 的方法定位与核心贡献

为避免本文与已有的 learning-for-search 或 post-hoc refinement 流程相混淆,我们在此进一步明确 LSL 的方法定位与核心贡献。用最直接的话来说,本文并不是提出一个“先由神经网络给出热图、再由搜索器做后处理”的松散两阶段流程;本文真正提出的是一个面向大规模 VRP 的闭环系统,其中学习负责生成对搜索友好的结构先验,而搜索又持续反过来改进学习目标本身。

LSL 试图解决什么问题? 本文关注的核心问题是:如何在超大规模 VRP 上同时实现可扩展性、全局建模能力和高解质量。许多学习型方法在规模增大后会遭遇显存或计算瓶颈,而另一类可扩展方法则往往依赖 partition 或 staged decomposition,以牺牲长程耦合为代价换取规模能力。与此同时,许多 learning + search 方法仍停留在 open-loop 设定中:搜索可以提升当前实例,却没有被系统性地转化为后续学习监督。

LSL 的核心定位是什么? LSL 是一个让 *Learning to Search* 与 *Searching to Learn* 相互促进的 closed-loop framework。Learning to Search 的含义是:模型学习的不是一次性直接吐出完整解,而是输出对后续搜索最有帮助的结构先验。Searching to Learn 则有两层含义。第一,搜索会把结构反馈回流给模型。第二,也是更关键的一点,搜索阶段回流的并不只是单个更优解。我们会保留并行启发式搜索得到的多条高质量解,并把它们共同投影到同一稀疏候选图上,形成逐行软目标分布。这个目标在结构稳定的节点上更集中,在结构不确定的节点上则保持更分散,从而让模型学会在哪里应当明确偏向,在哪里应当为后续搜索保留多个合理备选。

为什么这样的闭环此前难以真正成立? 一个原因来自架构本身:如果 backbone 过重,闭环无法运行在真正的大规模实例上;如果可扩展性只能通过 partition 获得,那么模型在进入搜索之前就已经丢失了关键的全局信息。这也是本文提出轻量全局稀疏骨干的根本原因。

另一个原因来自监督目标本身。许多已有方法隐式地把训练目标压缩为单一确定解,或鼓励模型输出几乎已经“隐式满足”组合优化约束的热图。这种监督有助于直接解码出可行解,但也会让模型过早变得过于确定,从而削弱其作为搜索先验的价值。与此不同,我们的监督目标来自并行搜索得到的多条高质量解,而不是单条路径。它不会对所有节点一视同仁:若多条高质量解在某个局部结构上高度一致,则对应目标分布会更尖锐;若存在多个合理候选,则目标分布会保持更平缓、更分散。这种保留结构不确定性的监督方式,正是让模型输出既有明显偏好、又不过早锁死搜索空间的关键。

本文的核心技术贡献 基于上述定位,本文的技术贡献可以概括为以下几点:

- **提出 LSL 闭环框架。**我们将大规模 VRP 的学习与搜索统一为一个双向反馈系统,而不是单向的模型加后处理流程。
- **提出面向超大规模实例的全局稀疏骨干。**我们设计了一个轻量、保持全局视野且不依赖硬分解的稀疏生成架构,使闭环能够在消费级 GPU 上真正运行起来。
- **提出由并行搜索诱导的多解软监督。**我们把多条高质量搜索结果转化为同一稀疏候选图上的逐行归一化软目标,使模型在结构稳定处学到更尖锐的偏好,在结构模糊处保留搜索所需的探索空间。
- **构建可持续自我改进的实际系统。**同一个闭环会持续把更强的搜索结果回流为更强监督,从而在大规模 VRP 上实现有效且可扩展的 self-refine。

总的来说,本文的真正贡献不是一个孤立的神经求解器,也不是把模型和启发式搜索简单串联起来;真正的贡献是一个可扩展的闭环系统,其中全局稀疏生成、搜索引导以及保留不确定性的监督目标被设计成协同工作的整体。

806 J LSL 的计算复杂度分析

807 对计算复杂度进行形式化分析，对于开展充分比较至关重要。下面给出详细分析：

808 稀疏骨干的复杂度

809 记 n 为 row-slot 表示中的行数， K 为每一行的候选宽度， d 为隐藏维度， L 为稀疏注意力层
810 数。与必须显式物化所有点对关系的稠密图模型不同，GSDT 只处理 $n \times K$ 的 row-slot 候
811 选结构。因此，主导性的注意力、投影和逐行 logit 计算都随稀疏支持规模而增长，而不是随
812 n^2 增长。忽略多头投影与前馈层的常数项后，神经骨干的时间复杂度可以概括为 $O(LnKd)$ ，
813 显存复杂度则为 $O(nK)$ 。

814 这与重型 constructive decoder 有何不同

815 在重型 constructive decoder 中，神经模型需要在一个不断增长或不断变化的部分解上反复
816 进行序列级决策，因此整个构造过程往往会表现出关于 n 的三次复杂度。LSL 避免了这一
817 成本，因为它在候选图上一次性预测稀疏先验，而不是强迫神经模型显式执行完整的构造轨
818 迹。

819 启发式精炼的复杂度

820 下游搜索后端同样直接运行在稀疏候选图上，而不是回到稠密的 $n \times n$ 图。其精确运行时间
821 取决于所选局部算子以及精炼预算 I ，但关键事实是：神经部分和搜索部分都被限制在稀疏
822 支持上。这正是完整 learning-search 闭环能够在超大规模实例和消费级 GPU 上真正落地
823 的原因。

824 K 可视化

825 在本节中，我们提供来自 TSPLIB 和 CVRPLIB 的图示示例，以展示学习到的稀疏先验
 826 引导下的代表性搜索轨迹。这里的图像应理解为若干中间精炼状态，而不是对某一种特定
 827 constructive decoder 的绑定说明。它们主要用来展示：在模型偏好的候选支持内，局部编
 828 辑如何逐步修复过长边、交叉和不良路径结构。

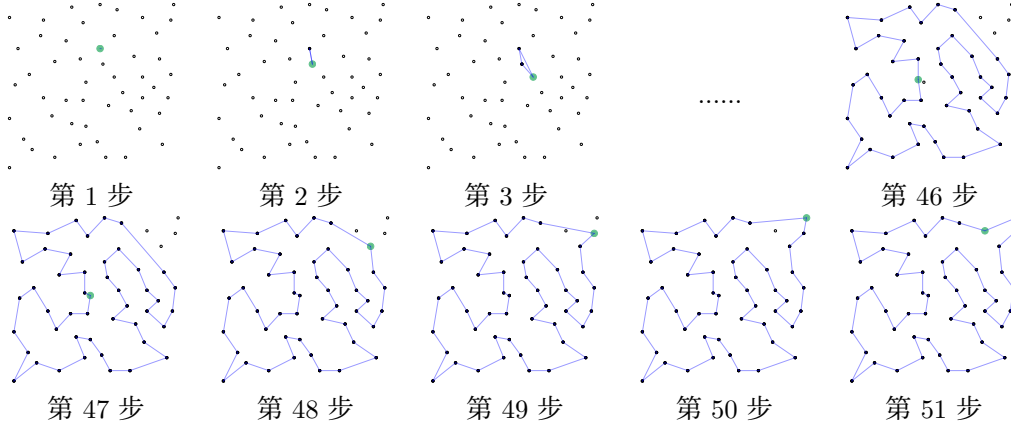


Figure 4: TSPLIB 实例“eil51”上的代表性精炼轨迹。绿色点表示当前步骤中正在被编辑的节点。空心点表示尚未确定的节点。黑点表示已经并入当前轨迹的节点。

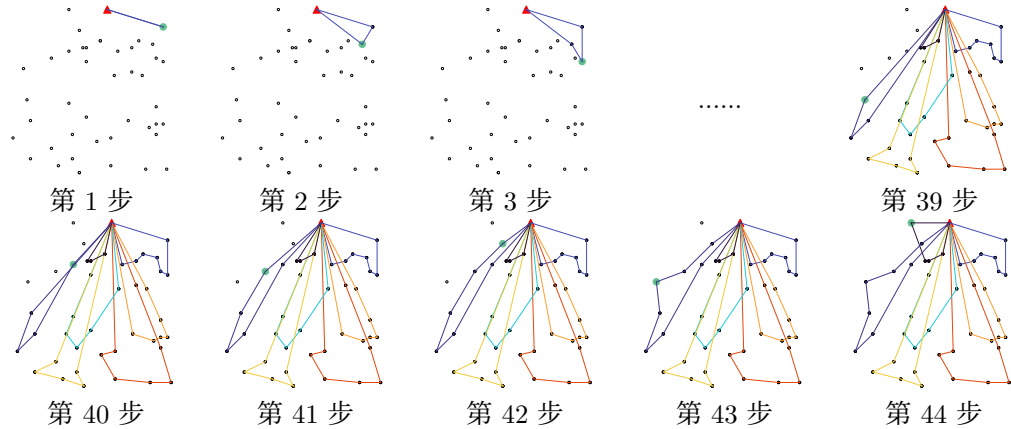


Figure 5: CVRPLIB 实例“A-n45-k7”上的代表性精炼轨迹。红色三角形表示仓库节点，绿色点表示当前步骤中正在被编辑的客户节点。空心点表示尚未确定的客户节点。黑点表示已经并入当前路径结构的客户节点。

829 L 更广泛的影响

830 本研究通过引入 LSL 这一面向大规模 VRP 的学习—搜索闭环框架，推动了神经组合优化
831 领域的发展。我们希望这一框架能够为后续关于可扩展、搜索感知型神经优化方法的研究提
832 供有价值的启发。作为一种通用的 VRP 学习求解范式，LSL 预计不会带来超出物流与路径
833 优化系统既有影响之外的特定负面社会影响。

834 M 许可

835 本工作中所使用代码与数据集的许可信息列于表 ??。

Table 13: 本工作使用的代码与数据集许可清单

资源	类型	链接	许可
LKH3 [?]	代码	http://webhotel4.ruc.dk/~keld/research/LKH-3/	仅供学术研究使用
HGS [?]	代码	https://github.com/chkwon/PyHygese	MIT 许可
Concorde [?]	代码	https://github.com/jvkersch/pyconcorde	BSD 3-Clause 许可
POMO [?]	代码	https://github.com/yd-kwon/POMO	MIT 许可
Att-GCN+MCTS [?]	代码	https://github.com/SaneLYX/TSP_Att-GCRN-MCTS	MIT 许可
LEHD [?]	代码	https://github.com/CIAM-Group/NCO_code/tree/main/single_objective/LEHD	MIT 许可
BQ [?]	代码	https://github.com/naver/bq-nco	CC BY-NC-SA 4.0
GLOP [?]	代码	https://github.com/henry-yeh/GLOP	MIT 许可
HTSP [?]	代码	https://github.com/Learning4Optimization-HUST/H-TSP	MIT 许可
DIMES [?]	代码	https://github.com/DIMESTeam/DIMES	MIT 许可
DIFUSCO [?]	代码	https://github.com/Edward-Sun/DIFUSCO	MIT 许可
UDC [?]	代码	https://github.com/CIAM-Group/NCO_code/tree/main/single_objective/UDC-Large-scale-CO-master	MIT 许可
ELG [?]	代码	https://github.com/gaocrr/ELG	MIT 许可
INViT [?]	代码	https://github.com/Kasumigaoka-Utaha/INViT	仅供学术研究使用
Neuropt [?]	代码	https://github.com/yining043/Neuropt	MIT 许可
TSPLib [?]	数据集	http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/	仅供非商业用途
CVRPLib [?]	数据集	http://vrp.gallos.inf.puc-rio.br/index.php/en/	仅供学术研究使用

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the closed-loop LSL framework, its sparse scaling motivation, and its empirical scope, and these claims are matched by the method and experiment sections.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The paper includes a “Conclusion, Limitation, and Future Work” section that discusses current dependence on fixed heuristic operators, tighter backbone-search co-design, and broader problem coverage.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in

favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is primarily algorithmic and empirical; it does not present theorem-proof style theoretical results that would require formal proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: The paper describes the method, datasets, budgets, metrics, and some implementation details, but the submission does not yet disclose a complete reproduction package for all reported training and evaluation results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The manuscript cites public baselines and datasets, but this submission does not include an anonymized release of the authors' code, generated assets, or a full supplementary reproduction package.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [No]

Justification: The experimental sections and appendix specify datasets, evaluation protocols, and several inference settings, but they do not yet enumerate all training hyperparameters and optimization details needed for a full understanding of every result.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- 997 • The experimental setting should be presented in the core of the paper to a level
998 of detail that is necessary to appreciate the results and make sense of them.
999 • The full details can be provided either with the code, in appendix, or as sup-
1000 plemental material.

1001 7. Experiment statistical significance

1002 Question: Does the paper report error bars suitably and correctly defined or other
1003 appropriate information about the statistical significance of the experiments?

1004 Answer: [No]

1005 Justification: The reported experimental tables provide point estimates for cost,
1006 gap, and runtime, but the paper does not report error bars, confidence intervals, or
1007 formal significance tests.

1008 Guidelines:

- 1009 • The answer [N/A] means that the paper does not include experiments.
- 1010 • The authors should answer [Yes] if the results are accompanied by error bars,
1011 confidence intervals, or statistical significance tests, at least for the experiments
1012 that support the main claims of the paper.
- 1013 • The factors of variability that the error bars are capturing should be clearly
1014 stated (for example, train/test split, initialization, random drawing of some
1015 parameter, or overall run with given experimental conditions).
- 1016 • The method for calculating the error bars should be explained (closed form
1017 formula, call to a library function, bootstrap, etc.)
- 1018 • The assumptions made should be given (e.g., Normally distributed errors).
- 1019 • It should be clear whether the error bar is the standard deviation or the stan-
1020 dard error of the mean.
- 1021 • It is OK to report 1-sigma error bars, but one should state it. The authors
1022 should preferably report a 2-sigma error bar than state that they have a 96%
1023 CI, if the hypothesis of Normality of errors is not verified.
- 1024 • For asymmetric distributions, the authors should be careful not to show in
1025 tables or figures symmetric error bars that would yield results that are out of
1026 range (e.g., negative error rates).
- 1027 • If error bars are reported in tables or plots, the authors should explain in the
1028 text how they were calculated and reference the corresponding figures or tables
1029 in the text.

1030 8. Experiments compute resources

1031 Question: For each experiment, does the paper provide sufficient information on the
1032 computer resources (type of compute workers, memory, time of execution) needed
1033 to reproduce the experiments?

1034 Answer: [No]

1035 Justification: The paper reports runtime numbers and notes consumer-GPU feasi-
1036 bility, but it does not provide a complete per-experiment hardware and memory
1037 specification for reproduction.

1038 Guidelines:

- 1039 • The answer [N/A] means that the paper does not include experiments.
- 1040 • The paper should indicate the type of compute workers CPU or GPU, internal
1041 cluster, or cloud provider, including relevant memory and storage.
- 1042 • The paper should provide the amount of compute required for each of the
1043 individual experimental runs as well as estimate the total compute.
- 1044 • The paper should disclose whether the full research project required more com-
1045 pute than the experiments reported in the paper (e.g., preliminary or failed
1046 experiments that didn't make it into the paper).

1047 9. Code of ethics

1048 Question: Does the research conducted in the paper conform, in every respect, with
1049 the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This is benchmark-based algorithmic research on routing optimization using synthetic and public problem instances, with no human-subject interaction or sensitive personal data, and the submission is anonymized.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper includes a short broader-impacts paragraph, but it does not yet provide a substantive discussion of concrete negative societal risks and mitigations beyond a brief statement that impacts are expected to be limited.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The work studies routing optimization and does not describe release of high-risk generative models, scraped datasets, or similarly misuse-prone assets.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: The paper cites the baseline methods and benchmark datasets and includes a dedicated “Licenses” section with a table listing the asset sources and their reported licenses or usage terms.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, [paperswithcode.com/datasets](#) has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[No\]](#)

Justification: The paper discusses bootstrapped large-scale training data and related outputs, but this submission does not include standalone documentation or an anonymized release package for newly introduced assets.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing, user studies, or any other research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human participants are involved in this work, so IRB review and participant-risk disclosure are not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not part of the proposed method, experiments, or scientific claims in the paper.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.